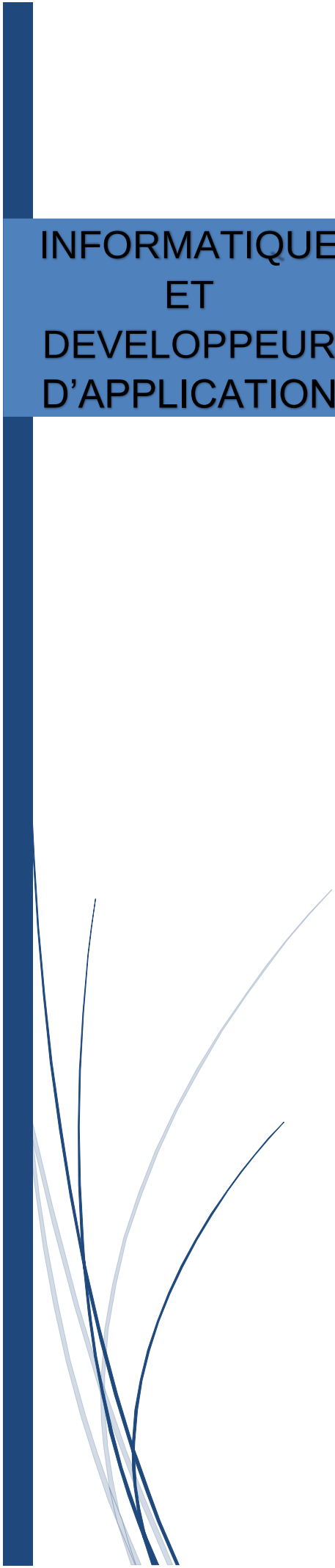




INFORMATIQUE
ET
DEVELOPPEUR
D'APPLICATION

LES BASES DES LANGAGES HTML5 & CSS3

Classe & Niveau : IDA1

Ministère de l'enseignement
Supérieur
0
Université Internationale Privée Abidjan

SOMMAIRE

<u>Chapitre 1 : Généralités sur le développement web</u>	1
1. Sites web	1
2. Les langages HTML et CSS	1
3. Les navigateurs web	2
4. Les éditeurs de texte	2
5. Les balises et attributs	2
6. Structure de base d'une page html5	3
7. Les commentaires html	4
<u>Chapitre 2 : Etude de quelques balises de structuration de contenu textuel</u>	5
1. Les balises de titre	5
2. La balise de paragraphe	6
3. La balise de saut de ligne 	6
4. La balise 	6
5. La balise 	6
6. La balise de marquage <mark></mark>	6
7. La balise <abbr></abbr>	6
8. La balise	7
9. La balise	7
10. Les balises de citation	8
<u>Chapitre 3: les liens hypertextes</u>	10
1. Les liens internes ou liens relatifs	10
2. Les liens externes ou liens absolus	11
3. Les liens pour envoyer un mail	11
4. Les liens de télécharger de fichier	11.
<u>Chapitre 4 : les images en html</u>	12
1. Les types d'images	12
2. L'insertion d'images : la balise 	13
2.1. Les attributs spécifiques à 	13
3. Liens sur images ou images cliquables	14
<u>Chapitre 5 : les listes html</u>	15
1. Liste non ordonnée	15
2. Listes ordonnées	15
3. Listes de définitions	16
4. Listes imbriquées	17
<u>Chapitre 6 : audio et vidéo</u>	18
1. Types de format	18
2. Insérer un élément audio	18
3. Insérer un élément vidéo	19
<u>Chapitre 7 : Les formulaires HTML</u>	21
1. Création d'un formulaire HTML	21
2. La balise <input /> et ses variantes	22
3. Zone de texte multi-lignes : la balise<textarea></textarea>	25
4. Zone de liste déroulante : La balise <select></select>	25
5. Finaliser un formulaire	26

5.1.	Regrouper les champs	26
5.2.	Sélectionner automatiquement un champ	26
5.3.	Rendre un champ obligatoire	26
Chapitre 8 : Créer des tableaux en HTML		28
1.	La structure générale d'un tableau simple en HTML	28
2.	Structurer un tableau (diviser un tableau HTML en différentes parties)	30
3.	Fusionner des cellules d'un tableau	32
Chapitre 9 : Pratique du langage css		34
I.	Différentes techniques utilisées pour écrire du code CSS	34
1.	Emplacement du code CSS	34
2.	Les Sélecteurs CSS	36
2.1.	Appliquer un style à toutes les occurrences d'un élément	36
2.2.	Appliquer un même style à plusieurs éléments	37
2.3.	Utilisation des attributs class et id pour certaines occurrences d'un élément donné	37
2.4.	Les sélecteurs avancés	39
II.	Formatage du texte en css	40
1.	Police du texte	40
2.	Taille des caractères	41
3.	Attributs des caractères	42
III.	Couleur des éléments et transparence	43
1.	Couleur du texte	43
2.	Couleur d'arrière-plan d'un élément	44
IV.	Personnalisation des listes	45
V.	Mise en forme des liens hypertextes	46
1.	changer la couleur de lien non visité	46
2.	Autres propriétés CSS applicables aux liens	46
VI.	Pseudo formats et apparences dynamiques	47
1.	Changer l'apparence d'un élément au survol	47
2.	Changer l'apparence d'un élément au moment du clic	47
3.	Changer l'apparence d'un élément lorsqu'il est sélectionné	48
4.	Changer l'apparence d'un lien lorsqu'il a été consulté (visité)	48
VII.	Gestion des images en css	49
1.	Image en arrière-plan d'une page	49
1.1.	Options disponibles pour l'image de fond	49
1.2.	Combiner les propriétés	50
2.	Plusieurs images de fond	50
3.	Empiler des images en CSS	51
VIII.	Ordures et ombres en css	52
1.	Ombrage des objets non textuels	52
2.	Ombrage des objets textuels	53
3.	Arrondir les coins d'un élément de type block	54
IX.	Le modèle des boites	56
1.	Les balises de type block et inline	56
2.	Les balises universelles	56
3.	Les dimensions	56
4.	Marges	57

5.	Centrer des blocs	58
X.	Tableaux et CSS	58
1.	bordures des colonnes	58
2.	fusionner et séparer des bordures	59
3.	espacer les bordures	59
4.	Arrière-plan des cellules	59
5.	Préciser l'emplacement de la légende	60
6.	Alignement dans les cellules	60

Chapitre 1 : Généralités sur le développement web

Le Web (ou WWW, ou *World-Wide Web*) est un ensemble de serveurs connectés à l'Internet et proposant des ressources (textes, images, sons, vidéos, etc.) dont le principal est le *document hypertexte*, un texte dans lequel certains mots, ou groupes de mots, sont des *liens (ancres)*, donnant accès à d'autres documents. Le langage qui permet de spécifier des documents hypertextes, et donc de fait le principal langage du Web, est HTML. Le langage html est à la base de tous les sites web.

1. Sites web

Un site Web est constitué d'une ou de plusieurs pages Web (fichier conçue pour être consultée avec un navigateur) reliées entre elles par des liens hypertexte ou hypermédia et accessibles par une adresse web ou URL (Uniform Resource Location). On distingue deux (2) catégories de sites web : les sites statiques et les sites dynamiques.

Les visiteurs d'un site web statique ne peuvent que voir simplement le contenu mais pas y participer. Pour les réaliser, seuls sont nécessaires les langages HTML, CSS et JavaScript pour ajouter du dynamisme mais uniquement du côté du navigateur (par exemple des animations graphiques).

Contrairement au site statique, un site dynamique offre un contenu qui peut évoluer dans le temps. Des programmes tournent du côté des serveurs, à l'arrière-plan pour générer le contenu du site. Ces programmes peuvent se servir de bases de données ou autres sources de données pour composer les pages qui seront affichées dans le navigateur. Les visiteurs pourront ainsi y participer. Pour réaliser un site dynamique, en plus des langages html, CSS et JavaScript, on a besoin de langages qui peuvent créer dynamiquement des pages, en analysant les requêtes des visiteurs pour ensuite fabriquer une réponse adaptée. Il existe plusieurs langages pour créer ces pages : PHP, Java, C#, Ruby, ...

***NB :** Dans le présent cours nous nous intéresserons à la création de site statique donc à l'apprentissage des langages HTML et CSS.*

2. Les langages HTML et CSS

Qu'il soit statique ou dynamique, le langage HTML est à la base de tout site web. Il est souvent utilisé conjointement avec le langage CSS. Le HTML (*HyperText Markup Language*) quant à lui permet de définir le contenu des pages. Entendez par là le texte, les images et les autres éléments qui doivent être affichés dans le navigateur, tandis que Le CSS (*Cascading Style Sheets*, aussi appelées *Feuilles de style*) permet de mettre en forme les éléments affichés (dimensions, bordures, couleurs, etc.) et de les agencer dans les pages (position, marges, etc.)

***NB :** Le HTML est à sa version 5 et le CSS à sa version 3.*

3. Les navigateurs web

Un navigateur web est un logiciel conçu pour surfer sur le web et en consulter les pages. Pour consulter une page ou toute sorte de ressource du web, l'utilisateur doit indiquer au navigateur l'url de la ressource ou de la page dans la barre de titre du navigateur. Celui-ci se connecte au serveur web hébergeant la ressource et la télécharge en utilisant généralement le protocole de communication **http**.

Il existe de nombreux navigateurs web pour toutes sortes de matériels (ordinateur personnel, tablette tactile, téléphone mobile) et pour différents systèmes d'exploitation (GNU/Linux, Windows, Mac OS, iOS et Android). Les plus utilisés à l'heure actuelle sont **Google Chrome, Mozilla Firefox, Internet Explorer/Edge, Safari, Opera**.

NB *il est conseillé d'installer plusieurs navigateurs afin d'observer le rendu de votre travail sur chacun d'eux.*

4. Les éditeurs de texte

Un éditeur de texte est un logiciel d'aide à la conception de pages html. Ils peuvent être classés dans 2 catégories :

- **Les éditeurs html WYSIWYG**, « What You See Is What You Get » : ce sont des éditeurs pour lesquels il n'est pas nécessaire de connaître le langage html ou CSS. Néanmoins il est bon d'avoir quelques notions dans ce domaine pour corriger des bugs générés fréquemment par ceux-ci. Il existe beaucoup d'éditeurs WYSIWYG de qualités très différentes et de prix très variés :
Exemple : Dreamweaver (un des meilleurs), BlueGriffon, NVU (de mozilla), KompoZer, etc.
- **Les éditeurs html de code et les IDE** : très utilisés par les professionnels, ils permettent d'afficher le code souvent avec de nombreuses aides :
Exemple: Bloc note, Visual Studio code, Sublime Text, Brackets, Atom, notepad++, NetBeans, etc.

5. Les balises et attributs

a. Balises

HTML est un langage de description du contenu d'une page web. Il fait appel à des balises (ou tag) pour structurer le contenu en délimitant des blocs qui seront amenés à contenir soit des paragraphes, soit des titres, soit différents types de médias (images, sons, vidéos), soit des contrôles de formulaires ou soit encore des liens hypertextes. Une balise débute par le signe « **inférieur à** », et finit par le signe « **supérieur à** » ("**<**" et "**>**").

On distingue deux types de balises : les balises « ouvertes et fermées » aussi appelées balises en paires et les balises « ouvrantes et fermantes » appelées balises orphelines ou balises seules.

- **Les balises ouvertes et fermées**: Les plus courantes, elles sont constituées d'une balise ouverte (ex :**<p>**) et d'une balise fermée (ex :**</p>**) indiquant le début et la fin d'un contenu. **On place entre les deux le contenu.**

Ex: **<a>****<a>**, **<p>****</p>**, **<h1>****</h1>**, ********, **<q>****</q>**, etc.

- **Les balises ouvrantes et fermantes** : un peu plus rare, elles servent le plus souvent à insérer un élément à un endroit précis (une image, saut de ligne, ...).
Ex : ou ,
 ou
, <hr/> ou <hr>

Nb le slash (/) à la fin n'est pas obligatoire.

Remarque

Lorsque les signes "<" et ">" sont omis, on parle non plus de balises mais d'éléments. Ex : p, h1, a, img, br, q, hr, em, ... sont des éléments.

b. Les attributs

Les attributs permettent d'apporter des informations supplémentaires aux balises. Ils se placent toujours **dans la balise** ouverte après « **l'élément** » pour les balises en paire et après « **l'élément** » pour les balises seules. Il est à noter que les attributs ont le plus souvent une valeur et qu'une balise peut comporter zéro, un ou plusieurs attributs. D'une manière plus générale, toutes les balises HTML sont construites sur les modèles :

<élément attr1="val" ... attrN="val"></élément>

Ou

<élément attr1="val" ... attrN="val">

Où

- « **élément** » est le nom d'un élément HTML (a, img, link, body, ...),
- Les « **attr** » sont des attributs HTML et
- les « **val** » sont les valeurs affectées à ces attributs.

6. Structure de base d'une page html5

Toute page web en HTML5 comprend au minimum le code qui est généralement généré automatiquement par la plupart des éditeurs de code HTML récents :

```
<!DOCTYPE html>
<html>
  <head>
    <meta charset="utf-8" />
    <title>Titre</title>
  </head>
  <body>
    <!-- ici se trouve tout ce qui doit s'afficher sur la page web-->
  </body>
</html>
```

Remarque:

- Il est conseillé d'indenter votre code car cela donne une plus grande lisibilité à votre code.
- Il est à noter aussi que les balises s'ouvrent et se ferment dans un ordre précis. Les balises doivent être fermées dans le sens inverse de leur ouverture.

Explications

- **<!DOCTYPE html>** est indispensable car c'est elle qui indique qu'il s'agit bien d'une page web HTML.
- La balise **<html> </html>** : balise principale du code, elle englobe tout le contenu de votre page.
- L'en-tête **<head></head>** contient les informations générales sur la page comme son titre, l'encodage (pour la gestion des caractères spéciaux), etc. Ces informations ne s'affichent pas sur la page, ce sont simplement des informations générales à destination de l'ordinateur.
Elle contient en outre :
 - **<meta charset="utf-8" />** : Cette balise indique l'encodage utilisé dans votre fichier « .html » c.-à-d. la façon dont le fichier est enregistré déterminant ainsi comment les caractères spéciaux vont s'afficher (accents, idéogrammes chinois et japonais, caractères arabes, etc.). Si vous avez un problème d'affichage des accents plus tard sur votre page web, c'est qu'il y a un problème avec l'encodage. Vérifiez que la balise indique bien UTF-8 et que votre fichier est enregistré en UTF-8.
 - **<title></title>** : Toute page doit avoir un titre qui décrit ce qu'elle contient. Il est conseillé de garder le titre assez court. Le titre ne s'affiche pas dans votre page mais en haut de celle-ci (souvent dans l'onglet du navigateur).
- Le corps **<body></body>** : il constitue la partie principale de la page. C'est donc notre espace de travail. C'est donc entre ces deux (2) balises que nous taperons le code de notre page. Tout ce que nous écrirons ici sera affiché à l'écran.

7. Les commentaires html

Comme dans tout langage informatique, un **commentaire** en HTML est un texte qui sert à se repérer dans un code source. Il n'est pas affiché, il n'est pas lu par l'ordinateur. Il sert aux personnes qui liront le code source de votre page. Pour insérer un commentaire en html, on utilise la balise.

<!-- ici on écrit le commentaire-->

NB : Un commentaire peut se mettre n'importe où au sein de votre code source : il n'a aucun impact sur votre page.

Chapitre 2 : Etude de quelques balises de structuration de contenu textuel

1. Les balises de titre

Dans une page web, les titres introduisent le contenu qui les suit et du point de vu référencement, après la balise <title>, les balises de titre sont probablement les plus importantes de l'ensemble du contenu d'une page web. Ils aident ainsi les lecteurs à comprendre ce qu'ils vont lire et à indiquer aux moteurs de recherche les informations les plus importantes que comportent votre page.

Dans une page html, les titres sont signalés par la présence des balises <h1></h1> à <h6></h6>. Ces balises n'ont pas le même poids pour les moteurs de recherche ; plus on augmente de niveau de titre, moins son contenu sera important pour les moteurs de recherche.

L'ordre d'apparition de ces balises dans le code source a de l'importance :

- Si le contenu de la page se hiérarchise (titre, sous-titre, sous-sous-titre, ...), il faudra utiliser les balises dans l'ordre suivant : h1, h2, h3, h4, h5, h6

```
<!DOCTYPE html>
<html>
  <head>
    <meta charset="UTF-8" />
    <title>CONTENU HIERARCHISE</title>
  </head>
  <body>
    <h1>La coupe d'Afrique des Nations </h1>
    <h2>Les pays qualifiés</h2>
    <h3>Par région géographique</h3>
    .....
  </body>
</html>
```

- Si le contenu de la page ne permet pas une utilisation hiérarchique des titres, optez pour un titre principal (h1) et plusieurs titres de même importance.

```
<!DOCTYPE html>
<html>
  <head>
    <meta charset="UTF-8" />
    <title>CONTENU NON HIERARCHISE</title>
  </head>
  <body>
    <h1>La coupe d'Afrique des Nations en Côte d'Ivoire </h1>
    <h2>Les villes hôtes</h2>
    <h2>Les stades retenus</h2>
    .....
  </body>
</html>
```

2. La balise de paragraphe <p></p>

En HTML l'une des balises les plus rencontrées, **<p>** marque le début d'un nouveau paragraphe qui se voit fermé par **</p>**. Son contenu est de type phrasé, c'est-à-dire qu'il contient du texte, potentiellement des sauts de ligne **
, mais aussi d'autres éléments de phrasé pouvant contribuer à la sémantique de ce texte, par exemple **, ****, **<abbr>**, **<ins>**, ****, **<q>**, etc.

Un autre paragraphe le suivant pourra correspondre à une nouvelle idée donc utilisation à nouveau d'un **<p>** et de **</p>**.

***NB :** Un paragraphe ne peut contenir un autre paragraphe.*

Remarque : Le style par défaut des navigateurs introduit un espace avant et après chacun des paragraphes, qui sont par nature des éléments de type bloc.

3. La balise de saut de ligne

Pour revenir à la ligne à l'intérieur d'un bloc de texte, comme un paragraphe, on utilise l'élément **
** (*line break* en anglais). Le texte qui suit le **
** est renvoyé à la ligne suivante. Il est noté que *
 doit être utilisé dans un paragraphe.*

4. La balise

La balise **** confère au texte une forte importance. Visuellement, il est représenté par un corps de police plus gras. Du contenu placé entre balises ** ** aura deux fois plus d'importance pour un moteur de recherche.

5. La balise

Une emphase est un procédé linguistique qui donne de l'importance ou une affectation pompeuse au discours. Une emphase peut être appliquée à une portion de texte avec ** **. Visuellement, les navigateurs afficheront le texte en italique.

6. La balise de marquage <mark> </mark>

Un texte contenu entre les balises **<mark> </mark>** est considéré comme marqué ou « surligné ». Il signale et met en valeur une portion du document sur laquelle on veut attirer l'attention, dans un contexte particulier. C'est l'élément du marqueur fluo.

7. La balise <abbr> </abbr>

Les abréviations sont de plus en plus courantes. On parle aussi de sigle et d'acronyme, pour lequel une balise **<acronym>** avait été prévue dans les versions précédentes de HTML, mais abandonnée puisque peu de rédacteurs faisaient réellement la différence entre ces trois catégories, qui dépendent les unes des autres :

- une abréviation est un simple raccourcissement de texte (ex : labo, info, etc.);
- un sigle est l'abréviation d'une locution dont on ne conserve que les initiales que l'on sépare en théorie d'un point (ex : « T.V.A. »taxe sur la valeur ajoutée)
- un acronyme est un sigle dont l'ensemble des lettres se lit ou se prononce comme un mot (ex : « ONU », « Sida », « Unesco »).

La balise **<abbr> </abbr>** est là pour nous aider à définir ces expressions, sans devoir les préciser à la suite du texte, d'une façon gênante pour la lecture. Pour cela cette balise prend un attribut **title** contenant la version texte étendue de l'abréviation ou de l'acronyme. Il sera notamment utilisé pour afficher une infobulle au survol, si le lecteur désire connaître l'explication de l'abréviation

Syntaxe d'utilisation de <abbr>

<abbr title="version abrégé étendu de l'abréviation">mot abrégé</abbr>

Exemple d'usage de paragraphe ,de saut de ligne, de strong, de em, de mark et de abbr

<!DOCTYPE html>

<html>

<head>

<meta charset="UTF-8" />

<title>Ma Coupe d'Afrique</title>

</head>

<body>

<h1>La coupe d'Afrique des Nations en Côte d'Ivoire </h1>

<p>

La prochaine coupe d'Afrique des Nations se tiendra en république de
côte d'ivoire <mark>du 13 Janvier au 11 Février</mark> 2024.

</p>

<p>

Elle opposera 24 nations issues des 4 coins de l'Afrique. A
savoir :
 -du centre
-du nord
-du sud
-de l'Ouest.

</p>

<h2>Les villes hôtes</h2>

<p>

La <abbr title="Coupe d'afrique des Nations">CAN</abbr> pour la toute
première sera disputée par 24 nations qui seront logés dans 5 villes
différentes : Abidjan, Bouaké, Korhogo, Yamoussoukro et San-Pedro.

</p>

</body>

</html>

8. Les balises à usage mathématique et chimique

- **La balise **

Un fragment de texte peut être placé en indice avec la balise (*subscript* en anglais). L'intérêt est aussi mathématique et chimique.

Syntaxe d'usage de <sub>

_{fragment de texte en indice}

- **La balise **

L'opposé de <sub> pour placer le texte en exposant est <sup> (*superscript* en anglais). L'intérêt est aussi mathématique et chimique, mais permet aussi de placer des références pour des notes en bas de page.

Exemple d'usage de <sup> et de <sub>

```
<!DOCTYPE html>
<html>
  <head>
    <meta charset="UTF-8" />
    <title>Formules mathématiques et chimiques</title>
  </head>
  <body>
    <h1>Rappels de quelques formules mathématiques et chimiques </h1>
    <h2>Formules mathématiques</h2>
    <p>
      La forme développée de  $(x + 2)^2$  est :  $x^2 + 4x + 4$  <br>
      Soit la suite numérique suivante :  $U_{n+1} = U_n + 4$ 
    </p>
    <h2>Formules chimiques</h2>
    <p>
      la formule chimique de l'eau est :  $H_2O$  et celle du dioxyde de
      carbone est  $CO_2$ 
    </p>
  </body>
</html>
```

9. Les balises de citation

Dans du contenu HTML, nous sommes parfois appelés à fait référence d'une tiers personne ou d'un auteur. HTML met à notre disposition des balises de citation à cet effet.

- **La balise <q></q>**

La balise **<q> </q>** est réservé aux citations courtes. La citation est encadrée automatiquement par des **guillemets** dans tous les navigateurs, sauf IE7 et versions inférieures. Il est à noter que les citations courtes se trouvent toujours dans un **<p></p>**.

Nb : Si nécessaire, Il est possible de faire référence à l'adresse à la source de la citation à l'aide de l'attribut **cite**.

```
<p>
  <q cite="Adresse de la source"> La citation </q>
</p>
```

- **La balise <blockquote></blockquote>**

Un bloc de citation **<blockquote></blockquote>** représente une section du document qui est extraite d'une autre source. Il peut contenir un ou plusieurs paragraphes, mais aussi des titres et des images.

Nb : Si nécessaire, Il est possible de faire référence à l'adresse à la source de la citation à l'aide de l'attribut **cite**.

```
<blockquote>
  <h1>Titre</h1>
  <p>Un paragraphe</p>
</blockquote>
```

- **la balise <cite></cite>**

L'élément **cite** représente le titre d'une œuvre, citée ou juste mentionnée dans le document. Cette catégorie englobe le titre d'un ouvrage, d'une chanson, d'une pièce de théâtre, d'un opéra, d'un film, d'une émission de télévision, d'un jeu, d'une œuvre d'art (peinture, sculpture), d'une exposition, ou plus couramment d'une autre page web, etc.

```
<p>
  <cite>Titre du document</cite>
</p>
```

Exemple d'usage de <q>

```
<!DOCTYPE html>
<html>
  <head>
    <meta charset="UTF-8" />
    <title>Les citations en HTML</title>
  </head>
  <body>
    <h1>L'exploration de l'espace</h1>

    <p>
      Un célèbre acteur Américain, dans le film <b><cite>Les larmes du
      soleil</cite></b> a dit : <b><q>Dieu a déjà quitté l'Afrique. </q></b>
    </p>

  </body>
</html>
```

Exemple de citation à l'aide de <blockquote>

```
<!DOCTYPE html>
<html>
  <head>
    <meta charset="UTF-8" />
    <title>Forum Berners Lee</title>
  </head>
  <body>
    <p>Tim Berners-Lee a écrit : </p>
    <blockquote cite="http://www.w3.org/People/Berners-Lee/Kids">
      <p>I just had to take the hypertext idea and connect it to the TCP and
      DNS ideas and -- ta-da! -- the World Wide Web.</p>
    </blockquote>

  </body>
</html>
```

Remarque

L'attribut **cite** peut être ajouté à cette balise pour faire référence à l'adresse de cette source. Il n'est pas rendu visuellement dans les navigateurs traditionnels.

Chapitre 3: les liens hypertextes

La magie du web tient en grande partie aux liens hypertexte : en définissant des liens sur vos pages, il est possible de renvoyer l'utilisateur vers une page de votre site ou vers une page d'un autre site.

Les liens se reconnaissent facilement sur une page : ils sont écrits par défaut, en bleu et soulignés et un curseur en forme de main apparaît lorsqu'on pointe dessus. Nous distinguons plusieurs types de liens mais d'une manière générale, ils sont tous définis avec la balise `<a>` `</a>` avec la syntaxe suivante:

`<a href="adresse de destination" > texte du lien</a>`

Où **adresse** est l'adresse de la page à atteindre et **texte** est le texte sur lequel l'utilisateur cliquera pour accéder à la page visée.

*NB : Quel que soit le type de liens créés, la seule chose qui change est la valeur de l'attribut **href**.*

1. Les liens internes ou liens relatifs

On désigne par lien interne, un lien crée entre 2 pages d'un même site. Nous avons trois cas à distinguer selon l'emplacement des 2 pages:

- 1er cas :

La page à partir de laquelle on fait un lien et celle vers laquelle on fait un lien se trouvent dans le même dossier (même niveau d'arborescence). Dans ce premier cas, il suffit de préciser le **nom** de la page dans l'attribut **href**.

Exemple

Soient 2 pages web **accueil.html** et **articles.html** situées dans un même dossier nommé « **mon projet** ». Nous désirons placer un lien depuis la page d'accueil qui renverra l'utilisateur sur notre page **articles.html** pour consulter la liste de nos articles en vente. Pour y parvenir, depuis **accueil.html**, il faut écrire depuis **accueil.html** :

<p>

<!-- Lien relatif vers une page nommée « articles.html » -->

Notre catalogue

</p>

- 2^e cas :

La page vers laquelle on souhaite faire un lien se trouve dans un sous dossier. Dans ce cas, il faudra en tenir compte et inclure le nom du sous-dossier dans la valeur de l'attribut **href**.

Exemple

Supposons que depuis notre dossier « mon projet », nous créons un sous dossier nommé « pages_secondaires » dans lequel nous plaçons le fichier articles.html. Pour faire maintenant un lien depuis accueil.html qui renverra l'utilisateur vers notre catalogue, il faudra écrire :

<p>

<!-- Lien vers un fichier dans le sous-répertoire (par rapport au document courant) -->

Notre catalogue

</p>

- **3è cas :**

La page vers laquelle on veut faire un lien se trouve dans un dossier parent. Dans ce cas, nous devons rajouter «../ » dans la valeur de l'attribut href.

Exemple

Supposons maintenant que nous désirons placer un lien dans le fichier « articles.html » se trouvant dans « pages_secondaires » qui renverra l'utilisateur sur la page d'accueil donc au fichier « accueil.html » se trouvant dans le dossier « mon_projet ».

CODE HTML

```
<p>
    <!-- Lien vers une page hébergée dans le répertoire parent -->
    <!--page cible est à un niveau au-dessus du document courant -->
    <a href="../accueil.html">Retour à la Page d'accueil</a>
</p>
```

2. Les liens externes ou liens absolus

Pour créer des liens externes, il suffit d'indiquer l'URL complète de la page vers laquelle on veut faire un lien en valeur de l'attribut **href**. On précisera le protocole (http://, ftp://, etc.)

Exemple

```
<p>
    <!-- Lien externe ou absolu vers facebook-->
    <a href="https://www.facebook.com/">facebook</a>
</p>
```

3. Les liens pour envoyer un mail

Pour envoyer un mail, on donne tout simplement une valeur de type

« **mailto : adresseMailDestinateur** » à l'attribut href. Il incombera au navigateur et au système d'exploitation de lancer l'application de messagerie installée pour prendre en charge ce lien et proposer une fenêtre de rédaction.

Exemple

```
<p>
    <!-- Lien pour adresse e-mail -->
    <a href="mailto:dm@gmail.com">clickez ici pour nous écrire</a>
</p>
```

4. Les liens de télécharger de fichier.

Pour cela, il va falloir placer le fichier à télécharger dans le même dossier que la page web à partir de laquelle on crée le lien. Ensuite, on crée un lien comme d'habitude en utilisant un chemin relatif avec le **nom du fichier** comme **valeur** de l'attribut **href**.

Exemple

```
<p>
    <!-- Lien pour télécharger un fichier -->
    <a href="courshtml.PDF">clickez ici pour télécharger le cours</a>
</p>
```

Chapitre 4 : les images en html

On n'envisage plus aujourd'hui un site sans images. Les sites grand public se doivent donc d'égayer leurs pages avec des illustrations photographiques ou simplement graphiques. L'abus d'images pouvant se révéler aussi nocif que leur absence en termes d'attractivité du site pour le visiteur, il appartient au concepteur d'effectuer un choix judicieux de ses illustrations. De plus, il faut encore tenir compte du poids des images car elles peuvent allonger le temps d'affichage complet d'une page si elle en contient beaucoup.

1. Les types d'images

Les navigateurs actuels n'acceptent qu'un nombre restreint de types d'images et il faudra vous limiter aux trois grands types les plus utilisés et enregistrer vos images selon les formats présentés ci-après, qui sont suffisants pour satisfaire tous les besoins d'un concepteur de sites :

- **GIF** : **G**raphics **I**nterface **F**ormat, ayant pour extension « **.gif** ». Ce format est limité à 256 couleurs et est donc déconseillé pour les photographies ayant une grande plage de teintes différentes. On l'utilisera en priorité pour insérer les petits éléments graphiques, icônes, des bandeaux publicitaires, petites images **animées**. Ce format GIF supporte un seul niveau de transparence (totalement opaque ou totalement transparent).
- **JPEG** : **J**oint **P**hotographic **E**xperts **G**roup, ayant pour extensions « **.jpeg** » ou « **.jpg** ». Ce format permet la création d'images ayant une grande plage de teintes différentes allant jusqu'à 16 millions de couleurs. Il est donc très adapté aux photographies réalistes. En contrepartie toutefois, les images JPEG ont habituellement un poids plus important en Ko, ce qui ralentit leur chargement et aussi *Il ne peut être rendu transparent.*
- **PNG** : **P**ortable **N**etwork **G**raphics, ayant pour extension « **.png** ». Ce format a été conçu comme alternative au format GIF qui faisait l'objet de droits d'auteurs. Le format PNG est utilisé pour les images de haute qualité avec niveaux de transparence et autres éléments graphiques.

Conseil pratique

À première vue, une image GIF ou PNG aura un poids plus faible qu'une image JPEG ; cependant, c'est la variété des teintes de l'image qui va influencer le poids final de l'image. Quand vous créez une image, essayez donc de l'enregistrer sous les trois formats. Vous serez parfois surpris par le poids réel des images obtenues et vous pourrez choisir la moins lourde, donc celle qui s'affichera le plus vite dans votre page. En ce qui concerne le temps d'affichage des images, vous pouvez tester vos pages en ligne pour vous mettre dans la situation de vos visiteurs car le temps d'affichage des images dans votre navigateur n'est pas le même que celui des visiteurs.

2. L'insertion d'images : la balise

La balise permet d'inclure des images dans une page web. Il s'agit d'un élément de type en ligne et généralement il doit être inclus directement dans un élément de type bloc comme un paragraphe.

Une image est insérée avec la balise **** en spécifiant les attributs **src** le chemin de l'image à inclure (sous la forme d'une adresse) et l'attribut **alt** l'éventuel texte de remplacement de l'image. La balise s'utilise de la manière suivante :

```

```

2.1. Les attributs spécifiques à

 peut avoir divers attributs facultatifs. Seuls **alt** et **src** sont obligatoires

- **src** : L'attribut src indique l'adresse à laquelle le navigateur doit la télécharger avant de l'afficher. Si seul src est indéfini, alors l'élément correspond au texte contenu dans alt est affiché.
- **alt** : Le texte alternatif d'une image. Il faut le considérer comme un contenu de repli lorsque l'image ne peut être visualisée. Soit parce qu'elle n'a pas été trouvée et téléchargée, soit parce que le navigateur ne peut comprendre son format, soit parce que l'utilisateur n'est pas en mesure de la voir.
 - Si l'attribut alt est vide, alors l'image est décorative.
 - Si l'attribut alt est absent et que l'image ne peut être interprétée ou affichée, le navigateur peut choisir d'exploiter l'attribut global title
- **Title** : titre de l'image (s'affiche en info bulle dans le navigateur)
- **Border** : la largeur de la bordure de l'image (en pixel)
- **Height** : la hauteur de l'image en pixel. Si aucune valeur spécifiée, alors l'image garde sa hauteur normale
- **Width** : largeur de l'image en pixel. Si aucune valeur spécifiée, alors l'image garde sa largeur normale
- **Align** : alignement de l'image pouvant prendre les valeurs **top**, **bottom**, **middle**, **left** ou **right**

Exemple d'usage de l'élément

```
<p>  
  <!-- Image dans le même répertoire que la page HTML -->  
    
  <!-- Image dans sous répertoire nommé mes_images-->  
    
  <!-- Image stockée à la racine du site web -->  
    
  <!-- Image située sur un autre site -->  
  <img src=http://www.antiquite.com/pyramide.jpg alt="la pyramide">  
</p>
```

3. Liens sur images ou images cliquables

Les liens hypertextes appliqués aux images revêtent les mêmes caractéristiques que les liens sur le contenu texte. L'intégralité de la surface de l'image est alors cliquable. Pour des raisons d'accessibilité, il est très fortement recommandé de toujours préciser un texte alternatif comme valeur de l'attribut alt.

**Exemple de liens associés à **

```
<p>  
  <a href="http://www.facebook.com" title="Cliquez pour vous connecter">  
      
  </a>  
</p>
```

Le lien, mène vers le site Facebook de l'entreprise Meta. La valeur de l'attribut title est affichée en infobulle au survol.

Chapitre 5 : les listes html

Les listes HTML sont prévues pour dresser des énumérations possédant une valeur sémantique. En HTML, on utilise les listes pour la conception de menus de navigation, qui ne sont en réalité que des énumérations de liens ou dans leur forme brute, pour mieux présenter du contenu pour un blog par exemple.

Il existe trois grands types de listes en HTML : les listes ordonnées, les listes non-ordonnées et les listes de définition.

1. Liste non ordonnée

Une liste ordonnée est une liste dont le contenu n'est pas spécialement trié et son ordre peut être changé sans altérer significativement son sens.

Pour créer une liste non-ordonnée, on utilise l'élément ul (abréviation de unordered list), donc la balise qui va contenir toute la liste et l'élément li (pour list item) donc utilisée pour créer chaque élément de la liste.

La syntaxe de création d'une liste non ordonnée est la suivante :

```
<ul>
  <li>Elément 1</li>
  .....
  <li>Elément N</li>
</ul>
```

Exemple de liste non ordonnée

```
<h1>les matières enseignées en IDA</h1>
<ul>
  <li>Algorithme et langage</li>
  <li>Développement Web</li>
  <li>Architecture des ordinateurs</li>
  <li>Systèmes d'exploitation</li>
</ul>
```

2. Listes ordonnées

Dans une liste ordonnée, il y a une relation de subordination, un ordre logique et naturel entre les éléments de la liste. Une liste ordonnée (*ordered list* en anglais) est équipée de puces numérotées automatiquement par le navigateur.

Pour créer une liste ordonnée, on utilise l'élément ol, donc la balise qui va contenir toute la liste et l'élément li donc la balise pour créer chaque élément de la liste. La syntaxe de création d'une liste non ordonnée est la suivante :

```
<ol>
  <li>Elément 1</li>
  <li>Elément 2</li>
  .....
  <li>Elément N</li>
</ol>
```

Exemple

<h1>les 3 plus grandes villes de Côte d'Ivoire</h1>

Abidjan

Bouaké

Korhogo

Attributs spécifiques à

- **Start** (nombre entier) : Valeur de départ pour la numérotation. Peut être négatif
- **Reversed** (pouvant prendre pour valeur **reversed** ou " " ou **sans valeur**) : Indique que la numérotation est descendante
- **Type** (pouvant prendre pour valeur **1** ou **a** ou **A** ou **i** ou **I**) : Type de numérotation (numérique classique, lettrée, chiffres romains)

Exemple avec utilisation des attributs sur ol

<h1>les 5 plus grandes villes de Côte d'Ivoire</h1>

<ol type= "a">

Abidjan

Bouaké

Daloa

Korhogo

Man

3. Listes de définitions

Les listes de définition sont utilisées pour définir des termes. Pour créer une liste de définition, il va nous falloir utiliser l'élément **dl** (pour definition list), l'élément **dt** (pour definition term) et l'élément **dd** pour la définition en soi.

NB : L'élément **dt** doit toujours être placé avant l'élément **dd**, c'est-à-dire le terme à définir avant sa définition.

La syntaxe de création d'une liste de définition est la suivante :

<dl>

<dt>Terme1 à définir</dt>

<dd>Définition 1 du terme1</dd>

.....

<dt>Terme2 à définir</dt>

<dd>Définition1 du terme2</dd>

<dd>Définition2 du terme2</dd>

.....

<dt> Terme3 à définir</dt>

<dd>Définition1 du terme3</dd>

<dd>Définition2 du terme</dd>

.....

</dl>

Exemple de liste de définition

```
<dl>
  <dt>ONU</dt>
  <dd>Organisation des Nations Unies</dd>
  <dt>UA</dt>
  <dd>Union Africaine</dd>
  <dt>PC</dt>
  <dd>Permis de Conduire</dd>
  <dd>Permis de Construire</dd>
</dl>
```

Remarque

Il est possible d'associer plusieurs termes à une définition ou même plusieurs définitions à un même terme.

4. Listes imbriquées

Le HTML offre la possibilité d'imbriquer les listes les unes dans les autres. Pour cela, il suffit de commencer une liste, puis d'en ouvrir une seconde à l'intérieur d'un élément de la première (on peut évidemment imbriquer plus de deux listes en répétant le même processus).

Voici une illustration, en imbriquant par exemple une liste non ordonnée à l'intérieur d'une autre liste non ordonnée :

```
<ul>
  <li>Sports collectives
    <ul>
      <li>Football</li>
      <li>Rugby</li>
      <li>Basket</li>
    </ul>
  </li>
  <li>Sports individuelles
    <ul>
      <li>Tennis</li>
      <li>Natation</li>
      <li>Saut en hauteur</li>
    </ul>
  </li>
</ul>
```

Chapitre 6 : audio et video

Les images ne sont pas les seuls éléments décoratifs d'une page web. À cet effet, nous disposons de multiples éléments multimédias qui vont des animations Flash à la vidéo, en passant par les fichiers sons. Avec le langage HTML, on peut incorporer ces fichiers multimédias de manière conforme en utilisant des balises appropriées.

1. Types de format

- **Les formats audio**

Les trois formats audio en compétition pour le Web sont :

- **MP3** (.mp3) : format de compression audio né en 1991 est à l'origine de l'essor de la musique numérique sur Internet. Malgré sa popularité, ce format est toujours soumis à des brevets. Lu par tous les lecteurs portables du marché, il reste dépendant de licences restrictives.
- **AAC** ou Advanced Audio Coding (.aac) : format sous brevets logiciels.
- **Vorbis** (.ogg) : libre de droits

- **Les formats vidéo**

Les trois formats vidéo en compétition pour le Web sont :

- **Ogg Theora** (.ogv) : On le retrouve majoritairement dans un conteneur Ogg. Il est issu directement du monde du libre, sans brevets.
- **WebM** (.webm) : Son utilisation est totalement gratuite, libre de tous droits.
- **H.264** (.mp4) : Aussi dénommé MPEG-4/AVC, le format H.264 est mis en avant par Apple pour Safari et ses plates-formes mobiles sous iOS. Internet Explorer 9 l'implémente également. Son avantage est de fournir une bonne qualité d'image, même à bas débit. Son principal inconvénient est qu'il dépend directement d'un partenariat nommé The Joint Video Team et de groupes possédant des intérêts économiques, dont le consortium MPEG. Étant soumis à des brevets logiciels il oblige le paiement de droits d'exploitation pour les programmes qui le produisent ou le lisent.

2. Insérer un élément audio

L'insertion d'un objet audio dans une page Web se réalise à l'aide de la balise `<audio></audio>`.

Tous les navigateurs n'étant pas compatibles avec les différents formats audio, pour diffuser de la musique sur le Web, il faudra proposer plusieurs formats du même fichier audio dans des balises **<source>** ainsi le navigateur utilisera le premier format avec lequel il est compatible.

La syntaxe de la balise audio est donc la suivante :

<audio>

`<source src="titre_fichier_audio.format1" >`

`<source src=" titre_fichier_audio.format2" >`

`<source src=" titre_fichier_audio.format3" >`

</audio>

Les attributs pour <audio>

- **Controls** : Il active la présence par défaut des contrôles visuels à l'écran (« Lecture », « Pause » et la barre de défilement). Il ne prend pas de valeur.
- **Loop** : Si loop présent, le média est lu indéfiniment en boucle. pas de valeur.
- **Autoplay** : déclenche la lecture automatiquement, aussitôt que le média est prêt. Il prend pour valeur autoplay ou on ne met rien comme valeur.
- **preload** : indique si le fichier média peut être préchargé dès le chargement du document HTML ou non. Il permet ainsi de réduire l'attente lorsque l'utilisateur choisit de lancer la lecture, en anticipant son téléchargement. Cet attribut peut prendre les valeurs :
 - **auto** (par défaut) : charger la totalité de tout fichier média
 - **metadata** : charge uniquement les métadonnées (durée, dimensions, etc.).
 - **none** : ne précharge rien. Utile si on veut éviter le gaspillage de bande passante sur votre site.

NB : L'attribut preload est ignoré si autoplay est présent, car il n'a alors plus aucune utilité.

Exemple

```
<audio controls loop autoplay="autoplay">
  <source src="gaou1er.ogg">
  <source src="gaou1er.mp3">
  <source src="gaou1er.aac">
</audio>
```

3. Insérer un élément vidéo

L'insertion d'un objet vidéo dans une page Web se réalise à l'aide de la balise **<video></video>**. Tous les navigateurs n'étant pas compatibles avec les différents formats vidéo, pour diffuser un contenu vidéo sur le Web, il faudra proposer plusieurs formats du même fichier vidéo dans des balises **<source>** ainsi le navigateur utilisera le premier format avec lequel il est compatible. La syntaxe de la balise vidéo est donc la suivante :

```
<video>
  <source src="titre_fichier_video.format1" >
  <source src=" titre_fichier_video.format2" >
  <source src=" titre_fichier_video.format3" >
</video>
```

Les attributs pour <video>

- **Poster** : indique une image de substitution à afficher en lieu et place de la vidéo avant la lecture
- **Controls** : Il active la présence par défaut des contrôles visuels à l'écran (« Lecture », « Pause » et la barre de défilement). Il ne prend pas de valeur.

- **Loop** : Si loop est présent, le média sera lu indéfiniment en boucle. pas de valeur.
- **Autoplay** : déclenche la lecture automatiquement, aussitôt que le média est prêt. Il prend pour valeur *autoplay* ou on ne met rien comme valeur.
- **preload** : indique si le fichier média peut être préchargé dès le chargement du document HTML ou non. Il permet ainsi de réduire l'attente lorsque l'utilisateur choisit de lancer la lecture, en anticipant son téléchargement. Cet attribut peut prendre les valeurs :
 - **auto** (par défaut) : charger la totalité de tout fichier média
 - **metadata** : charge uniquement les métadonnées (durée, dimensions, etc.).
 - **none** : ne précharge rien. Utile si on veut éviter le gaspillage de bande passante sur votre site.

NB : L'attribut preload est ignoré si autoplay est présent, car il n'a alors plus aucune utilité.
- **Width** : dimension en largeur de la zone d'affichage en pixel
- **Height** : dimension en hauteur de la zone d'affichage

Exemple

```
<video controls poster="Me.jpg" loop width="600" height="600">
  <source src="gaou1er.mp4">
  <source src="gaou1er.webm">
  <source src="gaou1er.ogv">
</video>
```

Chapitre 7 : Les formulaires HTML

Les formulaires introduits dans le HTML depuis ses plus anciennes versions sont l'élément essentiel qui permet l'interactivité entre un site et ses visiteurs.

Tout échange entre visiteur et serveur passe par un formulaire, dans lequel l'utilisateur peut saisir textes ou mots de passe, opérer des choix grâce à des boutons radio, des cases à cocher ou des listes de sélection, voire envoyer ses propres fichiers depuis le poste client. Il est donc important d'en maîtriser la création avec HTML, pour obtenir des formulaires présentables.

1. Création d'un formulaire HTML

Composés d'un ou de plusieurs champs de saisie, les formulaires sont délimités par les balises `<form>` et `</form>`. Pour être conforme au HTML, le document contenant le formulaire doit avoir la structure minimale suivante :

```
<form name="nom"
      enctype="application/x-www-form-urlencoded|multipart/form-data|text/plain"
      action="programme|action"
      method="get|post"
      autocomplete="on|off">
  ...
</form>
```

Où :

- L'attribut **name**="chaîne_de_caractères" attribue un nom au formulaire. Cet attribut est surtout utilisé pour accéder aux éléments du formulaire via un script JavaScript.
- L'attribut **enctype**="type_d'encodage" détermine le type d'encodage des données transmises au serveur. Sa valeur par défaut, "application/x-www-form-urlencoded", est utilisable dans la plupart des cas, à l'exception du transfert de fichiers du poste client vers le serveur, pour lequel elle doit être "multipart/form-data". Si l'attribut **action** a la valeur "mailto:", l'attribut **enctype** a pour valeur "text/plain" ou "text/html", selon que le contenu est envoyé à une adresse e-mail au format texte ou HTML.
- L'attribut **action** (obligatoire), Il désigne le fichier qui va traiter, sur le serveur, les informations saisies dans le formulaire. Il est recommandé que ce fichier soit présent dans le même répertoire que celui contenant le formulaire, mais ce n'est pas obligatoire. L'attribut **action** peut aussi avoir la valeur "**mailto:**", qui provoque l'envoi des données vers une adresse e-mail, qu'il faut préciser à la suite du mot **mailto**. Cette méthode ne peut servir qu'à envoyer des informations vers un mail mais pas vers une base de données.
- L'attribut **method**="POST|GET" détermine la méthode d'envoi des données vers le serveur. La méthode **GET**, qui est la méthode par défaut, présente l'inconvénient d'ajouter les données du formulaire à l'adresse URI du fichier qui les traite, ce qui les rend visibles par le visiteur. De plus, il existe une limite à la longueur des URI et donc à la quantité de données à transmettre. Ces problèmes ne se retrouvent pas avec la valeur **POST**, que vous utiliserez dans la plupart des cas.
- L'attribut **autocomplete** indique si le formulaire doit être complété automatiquement (on) ou non (off) en utilisant les données déjà saisies par l'utilisateur dans des champs similaires.

2. La balise `<input />` et ses variantes

La balise unique `<input />` permet de créer les composants classiques des formulaires et dont les aspects et les rôles sont très différents. La différenciation de ces composants s'effectue simplement en définissant la valeur de leurs attributs, et notamment de l'attribut **type**. L'attribut **name** est obligatoire, car c'est lui qui permet d'identifier les champs côté serveur et ainsi de récupérer leur contenu. Les sections qui suivent détaillent les éléments de type "text", "password", "email", "tel", "number", "search", "checkbox", "radio", "submit", "reset", "file" et "hidden"...

- **Zone de texte sur une seule ligne** : `<input type="text" />`

Ce champ crée un champ de saisie de texte d'une seule ligne. En plus de l'attribut **name**, vous pouvez apporter des précisions supplémentaires à l'aide des attributs suivants :

- **size**="nombre". Détermine la largeur de la zone en nombre de caractères.
- **maxlength**="nombre". Détermine le nombre maximal de caractères que l'utilisateur est autorisé à saisir.
- **value**="texte". Définit un texte par défaut tant que l'utilisateur ne l'a pas modifié. C'est cette valeur qui est transmise au serveur si l'internaute ne saisit aucun texte
- **placeholder**="texte", peut donner une indication sur le contenu du champ et qui disparaîtra dès que le visiteur aura cliqué à l'intérieur du champ.

Exemple:

Nom : `<input type ="text" name="nom" placeholder="Votre nom svp"`

- **Zone de mot de passe** : `<input type="password" />`

Crée un champ de saisie de mot de passe semblable à un champ texte mais dans lequel les caractères saisis sont invisibles et remplacés par des astérisques. Les attributs **size** et **maxlength** y jouent le même rôle que précédemment. En voici un

Exemple :

Mot de passe : `<input type ="password" name="code" size="10" maxlength="10"/>`

- **Zone de saisie de email** : `<input type="email" />`

Crée une zone de saisie d'Email. Le champ vous semblera *a priori* identique mais votre navigateur sait désormais que l'utilisateur doit saisir une adresse e-mail. Il peut afficher une indication si l'adresse n'est pas un e-mail.

Exemple :

Votre Email : `<input type="email" name="mail" />`

- **Zone de saisie de numéro de téléphone** : `<input type="tel" />`

Ce champ est dédié à la saisie de numéros de téléphone :

Exemple :

Téléphone : `<input type="tel" name="contact"/>`

Sur iPhone, par exemple, un clavier adapté s'affiche lorsqu'on doit remplir le champ.

- **Zone de saisie d'un nombre entier** : `<input type="number" />`

Ce champ permet de saisir un nombre entier : Le champ s'affichera en général avec des petites flèches qui permettront de changer la valeur. Le fonctionnement de ce champ peut être modifié avec les attributs suivants :

- **min** : valeur minimale autorisée.
- **max** : valeur maximale autorisée.
- **step** : c'est le « pas » de déplacement

Exemple :

Age : `<input type="number" min="18" max="99" step="1" name="age"/>`

- **Zone de saisie de date** : `<input type="date" />`

Différents types de champs de sélection de date existent :

- **date** : pour la date (05/08/1985 par exemple) ;
- **time** : pour l'heure (13:37 par exemple) ;
- **week** : pour la semaine ;
- **month** : pour le mois ;
- **datetime** : pour la date et l'heure (avec gestion du décalage horaire)
- **datetime-local** pour la date et l'heure (sans gestion du décalage horaire).

Exemple :

Date de naissance : `<input type="date" name="d_naissance" >`

- **Zone de recherche** : `<input type="search" />`

Crée un champ de recherche. Le navigateur décide ensuite comment afficher le champ de recherche. Ainsi, il peut ajouter une petite loupe au champ pour signifier que c'est un champ de recherche et éventuellement mémoriser les dernières recherches effectuées par le visiteur.

Exemple :

Recherche : `<input type="search" name="rech" />`

- **Case à cocher** : `<input type="checkbox" />`

Crée une case à cocher utilisée pour effectuer un ou plusieurs choix parmi ceux qui sont préétablis par le programmeur. L'attribut **value** contient le texte qui sera transmis au serveur si l'utilisateur coche la case. Il est obligatoire. On donne un nom différent à chacune case à cocher à l'aide de l'attribut **name**. L'attribut **checked="checked"** (pouvant être utilisé sans valeur) définit la case cochée par défaut.

Exemple :

```
<input type="checkbox" name="Foot" value="Football" />: Football<br>
<input type="checkbox" name="Cine" value="Cinema" checked /> : Cinema<br>
<input type="checkbox" name="Tele" value="Télévision" /> : Télévision<br>
<input type="checkbox" name="shopping" value="Shopping" /> : Shopping<br>
```

- **Bouton radio** : `<input type="radio" />`

Crée un bouton radio. Employé seul, un bouton radio peut servir à valider des choix. Employé en groupe, il implique, à la différence de cases à cocher, qu'un seul choix est autorisé. Dans ce cas, tous les boutons radio du groupe doivent avoir une même valeur pour l'attribut **"name"**. Le fait d'en activer un désactive celui qui l'était auparavant. L'attribut **checked="checked"** (pouvant être utilisé sans valeur) définit le bouton coché par défaut. L'attribut **value** joue le même rôle que pour les cases à cocher et est également indispensable.

Exemple d'utilisation de l'élément "radio" :

Débutant : `<input type="radio" name="niveau" value="débutant" checked/>`

Initié : `<input type="radio" name="niveau" value="initié"/>`

Intermédiaire : `<input type="radio" name="niveau" value="intermédiaire" />`

Professionnel : `<input type="radio" name="niveau" value="Professionnel" />`

- **Zone de sélection de fichier :** `<input type="file" />`

Champ permettant de sélectionner un fichier dans les mémoires de masse de l'ordinateur ce qui permet ainsi le transfert de fichiers du poste client vers le serveur. Cet élément crée un champ de saisie de même aspect qu'un champ de texte et un bouton de sélection de fichier permettant à l'utilisateur de choisir le fichier à transférer. L'attribut **name** est obligatoire. Vous pouvez utiliser en plus les attributs **size** limitant la taille de la zone de saisie, et plus particulièrement l'attribut **accept**, qui définit le ou les types de fichier acceptés en transfert. Par sécurité, l'utilisation de cet attribut est recommandée car il permet de limiter le transfert à certains types de fichiers bien précis et de refuser les autres.

Exemple

Dans cet exemple, le serveur n'accepte que le transfert de fichiers ayant les extensions **.gif** ou **.jpeg** et refuse tout autre type:

```
<input type="file" name="monfichier" accept="image/gif,image/jpeg" size="30"/>
```

- **Zone de champ caché :** `<input type="hidden" />`

Champ caché. Bien qu'il n'apparaisse pas dans le formulaire, son contenu est envoyé au serveur lorsque l'utilisateur clique sur le bouton Submit.

Les informations sont contenues dans l'attribut **value**. L'attribut **name** est obligatoire.

Exemple: `<input type="hidden" name="Taille_Max_Fichier" value="20000"/>`

- **Bouton d'envoi de données au serveur :** `<input type="submit" />`

Crée un bouton sur lequel l'utilisateur doit cliquer pour déclencher l'envoi des données de tout le formulaire vers le serveur.

Il est indispensable d'avoir au moins un bouton d'envoi par formulaire, mais il est possible d'en utiliser plusieurs. Le clic sur l'un de ces boutons est alors analysé par le script désigné par l'attribut **action** de l'élément `<form>`. Cela permet d'effectuer des tâches spécialisées en fonction de la valeur associée à chaque bouton grâce à son attribut **value**. C'est le contenu de l'attribut **value** qui constitue le texte visible du bouton dans le formulaire. L'attribut **name** n'est a priori pas utile, en particulier s'il n'y a qu'un seul bouton d'envoi.

Exemple :

```
<input type="submit" value="Envoyer" />
```

- **Bouton de réinitialisation du formulaire :** `<input type="reset" />`

Crée un bouton de réinitialisation du formulaire et non d'effacement systématique, comme on le croit souvent. Si les éléments du formulaire ont des attributs **value** qui définissent des valeurs par défaut, ce sont ces valeurs qui apparaissent au démarrage de la page et qui sont réaffichées si l'utilisateur clique sur le bouton **reset**. Le contenu de l'attribut **value** du bouton d'effacement constitue le texte visible du bouton dans le formulaire.

Exemple:

```
<input type="reset" value="Effacer" />
```

3. Zone de texte multi-lignes : la balise<textarea></textarea>

L'élément <textarea> crée un champ de saisie de texte permettant la saisie sur plusieurs lignes. Cet élément comporte une balise de fermeture </textarea> et un contenu de texte par défaut affiché dans la zone de texte.

Les attributs **cols** et **rows**, qui donnent respectivement le nombre de colonnes et de lignes de la zone de texte, doivent être définis.

Exemple:

```
<textarea name="commentaire" cols="15" rows="8" placeholder="Ici un  
commentaire" >  
</textarea>
```

Cet exemple crée une zone de saisie de texte de 15 colonnes et 10 lignes au maximum.

4. Zone de liste déroulante : La balise <select></select>

Crée une liste de sélection d'options (liste déroulante) parmi lesquelles l'utilisateur fait un choix, chaque option devant être définie par un élément <option> séparé.

L'élément <select> a la structure suivante :

```
<select name="monDeLaListe">  
  <option value="valeur"> Texte choix 1</option>  
  <option value="valeur"> Texte choix 2</option>  
  <option value="valeur"> Texte choix 3</option>  
</select>
```

- La balise <select></select> comporte les attributs suivants :
 - **name**="nom_select". Obligatoire. Donne le nom de la liste.
 - **multiple**="multiple". Autorise la sélection de plusieurs options simultanément.
 - **size**="Nombre". Détermine le nombre de choix visibles simultanément. Par défaut, sa valeur est 1.
- La balise <option> comporte les attributs suivants :
 - **value**="chaîne". Obligatoire. Définit la valeur transmise au serveur si l'option est sélectionnée.
 - **selected**="selected". Définit l'option qui est sélectionnée par défaut dans la liste si l'utilisateur ne fait pas de choix.

NB : Vous pouvez grouper vos options avec la balise <optgroup> </optgroup>.

Cette dernière prend l'attribut **label** pour donner un nom à chaque groupe d'optio

Exemple :

```
<form method="post" action="traitement.php">
  <select name="pays">
    <optgroup label="Afrique">
      <option value="Côte d'Ivoire">Côte d'Ivoire</option>
      <option value="Sénégal">Sénégal</option>
    </optgroup>
    <optgroup label="Amérique">
      <option value="canada">Canada</option>
      <option value="USA">Etats-Unis</option>
    </optgroup>
    <optgroup label="Asie">
      <option value="Chine">Chine</option>
      <option value="Japon">Japon</option>
    </optgroup>
    <optgroup label="Europe">
      <option value="France">France</option>
      <option value="Espagne">Espagne</option>
    </optgroup>
  </select>
</form>
```

5. Finaliser un formulaire

5.1. Regrouper les champs

Lorsque le formulaire comporte beaucoup de champs, il peut être utile de les regrouper au sein de plusieurs balises **<fieldset></fieldset>**.

Chaque **<fieldset>** peut contenir une légende avec la balise **<legend></legend>**.

5.2. Sélectionner automatiquement un champ

Il est possible de placer automatiquement le curseur dans l'un des champs de votre formulaire avec l'attribut **autofocus**. Ainsi dès que le visiteur chargera la page, le curseur se placera dans ce champ (un seul champ à la fois).

5.3. Rendre un champ obligatoire

Il est possible de faire en sorte qu'un ou plusieurs champs soient obligatoires en leur donnant l'attribut **required** (peut-être plusieurs fois).

Exemple :

L'exemple fournit le code html d'un formulaire contenant divers éléments

```
<!DOCTYPE html>
<html>
  <head>
    <meta charset="utf-8" />
    <title>Formulaire traité par PHP</title>
  </head>
  <body>
    <form action="cible.php" method="post">
      <fieldset>
        <legend>Vos coordonnées</legend>
        Nom : <input type="text" name="nom" placeholder="votre nom"
required/><br />
        Prénom :<input type="text" name="pren" placeholder="votre
prénom" required /><br />
        Mail : <input type="email" name="mail" placeholder="votre mail"
required /><br />
        Mot de passe: <input type="password" name="mdp"
maxlength="10" /> <br />
        <input type="radio" name="sexe" value="homme" /> Homme<br />
        <input type="radio" name="sexe" value="femme" /> Femme<br/>
        <select name="pays" size="1">
          <option value="ci"> Côte d'Ivoire</option>
          <option value="Canada"> Canada</option>
        </select><br />
      </fieldset>
      <fieldset>
        <legend>Vos loisirs</legend>
        <input type="checkbox" name="tel" value="Télé" /> Télévision<br/>
        <input type="checkbox" name="foot" value="Football" /> Football<br/>
        Description: <br />
        <textarea name="desc" cols="50" rows="5"></textarea><br /><br />
      </fieldset>
      <input type="reset" value="Effacer" />
      <input type="submit" value="Envoyer" /><br /><br />
    </form>
  </body>
</html>
```

Chapitre 8 : Créer des tableaux en HTML

Dans une page HTML, il est possible de réaliser une présentation d'informations de type textuelle ou graphique sous la forme de tableaux. De la même manière que dans un tableur Excel, un tableau créé en HTML est composé de lignes et de colonnes. À l'intersection de ces dernières se trouve une cellule du tableau dont le contenu peut être constitué aussi bien d'éléments de type bloc (titres, paragraphes, divisions, listes), d'éléments en ligne tels que des images, ou encore d'éléments de formulaire. Nous verrons qu'à partir d'une grille de base, il est possible de fusionner plusieurs cellules voisines, ce qui permet de créer des tableaux asymétriques très variés. Les tableaux ont souvent été utilisés pour élaborer des mises en page de documents sur plusieurs colonnes. Même si nous étudions dans ce chapitre le procédé permettant de réaliser des présentations variées, il faut savoir que cette tendance est actuellement abandonnée au profit de l'utilisation des propriétés

CSS de positionnement qui permettent de procéder aux mêmes types de réalisations d'une manière plus élégante et moins rigide. Nous retrouvons ici la tendance permanente en HTML de séparation du contenu et de la présentation. Les tableaux n'en continuent pas moins d'être des éléments essentiels d'une page HTML pour la présentation des données.

1. La structure générale d'un tableau simple en HTML

Un tableau est défini à l'aide de la balise **<table>**. Un tableau est composé de plusieurs lignes définies avec la balise **<tr>**. Chaque ligne contient une ou plusieurs cellules définies :

- avec la balise **<th>** s'il s'agit d'une cellule de titre ;
- avec la balise **<td>** s'il s'agit d'une cellule contenant une donnée.

Il est aussi possible d'ajouter un titre à nos tableaux à l'aide de la balise **<caption>**

La structure générale d'un tableau est donc la suivante :

```
<table>  
  <caption>Titre du tableau</caption>  
  <tr>  
    <th>Titre colonne 1</th>  
    <th>Titre colonne 2</th>  
  </tr>  
  <tr>  
    <td>Cellule 1 ligne 1</td>  
    <td>Cellule 2 ligne 1</td>  
  </tr>  
  <tr>  
    <td>Cellule 1 ligne 2</td>  
    <td>Cellule 2 ligne 2</td>  
  </tr>  
</table>
```

Exemple de tableau élémentaire

```
<!DOCTYPE html >
<html>
  <head>
    <title>Visiteurs</title>
    <meta charset="utf-8" />
  </head>
  <body >
    <table>
      <caption>Liste des Présidents élus démocratiquement de la CI</caption>
      <tr>
        <th> Nom et Prénoms</th>
        <th>Période </th>
      </tr>
      <tr>
        <td> Félix Houphouët Boigny</td>
        <td> De 1960 à 1993 </td>
      </tr>
      <tr>
        <td> Henri Konan Bédié</td>
        <td> De 1993 à 1999 </td>
      </tr>
      <tr>
        <td> Koudou Laurent Gbagbo </td>
        <td> De 2000 à 2010 </td>
      </tr>
      <tr>
        <td> Alassane Dramane Ouattara </td>
        <td> De 2010 à Aujourd'hui </td>
      </tr>
    </table>
  </body>
</html>
```

Remarque :

Par défaut en HTML n'a pas de bordure. Il faudra donc utiliser du CSS pour mettre en forme vos tableaux et éviter de les faire en HTML à l'aide d'attributs qui sont pour la part obsolètes.

2. Structurer un tableau (diviser un tableau en différentes parties)

Si vous créez un tableau long, il sera certainement préférable de commencer à l'organiser en le divisant en plusieurs sous-parties.

On peut diviser un tableau en trois sous-parties : une partie d'en-tête (header), un corps de tableau (body) et un pied (footer). Ces trois parties sont matérialisées en HTML par les éléments **thead**, **tbody** et **tfoot**.

- La balise **<thead>** représente un en-tête de tableau, utilisé en conjonction avec **<tbody>** et **<tfoot>**. Il doit être placé après **<caption>** et **<colgroup>** s'il y a lieu, et avant **<tbody>** ou **<tfoot>**. Son contenu est constitué d'enfants **<tr>**, eux-mêmes constitués d'un ou plusieurs **<th>**. La spécification HTML n'autorise pas d'y placer des cellules classiques **<td>**, car il ne s'agit pas de cellules pouvant représenter une tête de tableau.
- La balise **<tfoot>** représente un pied de tableau, utilisé en conjonction avec **<thead>** et **<tbody>**. On peut y placer **zéro** ou **plusieurs** **<tr>**.
Attention : **<tfoot>** doit apparaître **avant** **<tbody>** dans le code afin de fournir au navigateur.
- La balise **<tbody>** représente un corps de tableau, utilisé en conjonction avec **<thead>** et **<tfoot>**. Il regroupe les lignes **<tr>** qui contiennent les données du tableau.

```

<!DOCTYPE html >
<html>
  <head>
    <title>Visiteurs</title>
    <meta charset="utf-8" />
  </head>
  <body >
    <table>
      <caption>Liste des Passagers</caption>
      <thead>
        <tr>
          <th> Nom et Prénoms</th>
          <th>Age </th>
        </tr>
      </thead>
      <tfoot>
        <tr>
          <th> Nom et Prénoms</th>
          <th>Période </th>
        </tr>
      </tfoot>
      <tbody>
        <tr>
          <td> Félix Houphouët Boigny</td>
          <td> De 1960 à 1993 </td>
        </tr>
        <tr>
          <td> Henri Konan Bédié</td>
          <td> De 1993 à 1999 </td>
        </tr>
        <tr>
          <td> Koudou Laurent Gbagbo </td>
          <td> De 2000 à 2010 </td>
        </tr>
        <tr>
          <td> Alassane Dramane Ouattara </td>
          <td> De 2010 à Aujourd'hui </td>
        </tr>
      </tbody>
    </table>
  </body>
</html>

```

3. Fusionner des cellules d'un tableau

Les cellules d'un tableau peuvent être fusionnées horizontalement et/ou verticalement en utilisant respectivement les attributs **colspan** et **rowspan**. Chacun de ces deux attributs accepte un nombre entier en valeur qui indique le nombre de cellules qui doivent être collées entre elles. Ces attributs s'appliquent aux balises <td> et <th> et s'utilisent de la manière suivante :

Par exemple,

- l'instruction suivante fusionne horizontalement la cellule courante avec la cellule suivante : <td colspan="2">contenu</td>

Ou encore

- l'instruction suivante fusionne verticalement la cellule courante avec les deux cellules suivantes : <td rowspan="3">contenu</td>

- **Fusionner les lignes d'un tableau (à la verticale) : attribut rowspan**

Travail à faire : reproduire le tableau ci-dessous

Titre du film	Game of Thrones	Lucky Luke, seul contre tous	Robin des bois
Pour Enfants ?	NON	OUI	Pour toute la famille !
Pour Adolescents ?	OUI		
Pour Adultes ?		NON	

```
<!DOCTYPE html >
<html>
  <head>
    <title>Visiteurs</title>
    <meta charset="utf-8" />
  </head>
  <body>
    <table border ="2">
      <caption>Films et Générations</caption>
      <tr>
        <th>Titre du film</th>
        <td>Game of Thrones</td>
        <td> Lucky Luke, seul contre tous </td>
        <td> Robin des bois </td>
      </tr>
      <tr>
        <th><td>Pour Enfants ?</th>
        <td align="center">NON</td>
        <td rowspan="2" align="center">OUI</td>
        <td rowspan="3" align="center">Pour toute la famille !</td>
      </tr>
      <tr>
        <th> Pour Adolescents ?</th>
        <td rowspan="2" align="center"> OUI</td>
      </tr>
      <tr>
        <th>Pour Adultes ?</th>
        <td align="center">NON</td>
      </tr>
    </table>
  </body>
</html>
```

- **Fusionner les colonnes d'un tableau (à l'horizontal) : attribut colspan**

Travail à faire : reproduire le tableau ci-dessous

Titre du film	Pour enfants ?	Pour adolescents ?	Pour adultes ?
Game of Thrones	NON	OUI	
Robin des bois	OUI		NON
lucky luke, seul contre tous	Pour toute la famille !		

CODE HTML

```
<!DOCTYPE html >
<html>
  <head>
    <title>Visiteurs</title>
    <meta charset="utf-8" />
  </head>
  <body >
    <table border ="2">
      <caption>Films et Générations</caption>
      <tr>
        <th>Titre du film</th>
        <td>Game of Thrones</td>
        <td> Lucky Luke, seul contre tous </td>
        <td> Robin des bois </td>
      </tr>
      <tr>
        <th>Pour Enfants ?</th>
        <td align="center">NON</td>
        <td rowspan="2" align="center">OUI</td>
        <td rowspan="3" align="center">Pour toute la famille !</td>
      </tr>
      <tr>
        <th> Pour Adolescents ?</th>
        <td rowspan="2" align="center"> OUI</td>
      </tr>
      <tr>
        <th>Pour Adultes ?</th>
        <td align="center">NON</td>
      </tr>
    </table>
  </body>
</html>
```

Chapitre 9 : Pratique du langage css

Les feuilles de style en cascade généralement appelées CSS forment un langage informatique qui décrit la présentation des documents html et xml. Le langage CSS est donc utilisé pour mettre en forme les éléments affichés (dimensions, bordures, couleurs, etc.) et les agencer dans les pages (position, marges, ancrage, etc.).

Le principe de base du langage CSS est élémentaire : on affecte un ensemble de couples propriété-valeur à une ou plusieurs balises HTML.

I. Différentes techniques utilisées pour écrire du code CSS

1. Emplacement du code CSS

Le code CSS peut être écrit:

- Directement dans les balises concernées;
- Dans l'en-tête du document html (on parle de feuille de styles interne);
- Dans un fichier séparé (on parle de feuille de styles externe).

1.1. Styles CSS directement dans les balises

Pour définir des propriétés CSS dans une balise, on utilise l'attribut **style** comme suit:

<Elément style="propriété1: valeur1; propriétéN: valeurN;">

Où :

- **Elément** est un nom d'un élément html : p, em, ou h1, par exemple.
- **propriété1** à **propriétéN** sont des propriétés de style de la balise.
- **valeur1** à **valeurN** sont les valeurs affectées aux propriétés.

Exemple : Pour affecter la couleur jaune à l'arrière-plan d'un élément p, vous utiliserez le code suivant :

<p style="background-color:yellow;" > Ce texte a un arrière-plan jaune </p>

NB : Cette technique n'est pas optimale, car elle est limitée au seul élément qui l'utilise.

1.2. Feuille de styles interne

Pour étendre la portée de ces définitions, vous pouvez les regrouper dans l'en-tête du document HTML, entre les balises <style> et </style>. Voici la syntaxe permettant d'affecter des propriétés à une balise :

```
<head>
  <style>
    Élément {
      propriété1:valeur1;
      .....
      propriétéN:valeurN ;
    }
    Élément {
      propriété1:valeur1;
      .....
      propriétéN:valeurN ;
    }
    .....
  </style>
</head>
```

Où :

- **élément** est un nom d'élément : p, table, form ou strong, par exemple.
- **propriété1** correspond aux propriétés de style de l'élément.
- **valeur1** correspond aux valeurs affectées aux propriétés.

Exemple :

Pour définir dans tout le document un arrière-plan jaune pour les éléments p et pour utiliser la police Verdana corps 18 dans tous les titres h2, vous utiliserez le code suivant :

```
<!DOCTYPE html>
<html>
  <head>
    <meta charset="UTF-8" />
    <title>Une feuille de styles interne</title>
    <style>
      p {
        background-color:yellow;
      }
      h2 {
        font-family:Verdana; font-size: 18px;
      }
    </style>
  </head>
  <body>
    <h2>Introduction</h2>
    <p>la beauté d'un site web dépend du CSS</p>
  </body>
</html>
```

NB : Cette technique n'est pas optimale, elle est limitée au seul document qui l'utilise.

1.3. Feuille de styles externe

Pour étendre encore le champ d'action des styles définis dans l'en-tête d'un document, vous pouvez les stocker dans une feuille de styles externe (un fichier qui a pour extension **.css**). Dans ce cas, plusieurs pages HTML pourront utiliser cette feuille de styles afin d'obtenir une même mise en forme et ou une même mise en page.

Pour relier une feuille de styles externe à un document HTML, vous utiliserez la balise **<link/>** : **<link rel="stylesheet" href="nomFeuilleDeStyle.css" />** dans l'entête du fichier html.

Exemple : Supposons que :

- Notre feuille de style nommé « **style.css** » contienne le code CSS ci-après:

```
p {
  background-color:mocasin;
}
h1 {
  font-family:Verdana; font-size: 18px;
}
```

- Et que le document HTML qui fait référence à « style.css » est le suivant :

```
<!DOCTYPE html>
<html>
  <head>
    <meta charset="UTF-8" />
    <title>Une feuille de styles externe</title>
    <link rel="stylesheet" href="style.css" />
  </head>
  <body>
    <h1>présentation du html</h1>
    <p>le html est le langage de fait du web. </p>
  </body>
</html>
```

Conseil pratique :

Dans la mesure du possible, et en particulier si votre projet consiste à définir un site Web composé de plusieurs pages, il est conseillé de créer une feuille de styles externe et d'y faire référence dans chaque page avec une balise <link>. Ainsi, vos pages auront tout naturellement la même apparence.

De plus, si vous voulez changer certaines caractéristiques du site (la couleur d'arrière-plan, la hauteur de la bannière, la police des titres h2, ou n'importe quelle autre propriété), vous n'aurez qu'un seul fichier à modifier : la feuille de styles externe. Les modifications seront automatiquement reportées sur toutes les pages du site.

2. Les Sélecteurs CSS

On appelle sélecteur un ensemble de techniques permettant d'atteindre un élément c.-à-d. appliquer un style à une balise situé dans le code source de notre page html.

2.1. Appliquer un style à toutes les occurrences d'un élément

Si vous voulez modifier les caractéristiques de toutes les occurrences d'un élément (img, a, p, table, etc.) dans une page, indiquez le nom de l'élément concerné et définissez autant de couples propriété-valeur que nécessaire. Chaque propriété est suivie du symbole « deux-points » (:) puis de la valeur correspondante. Enfin, chaque ligne se termine par un point-virgule (;).

Schématiquement, une feuille de style CSS ressemble donc à cela :

```
Elément{
  Propriété :valeur ;
  Propriété :valeur ;
  Propriété :valeur ;
}
Elément{
  Propriété :valeur ;
  Propriété :valeur ;
  Propriété :valeur ;
}
```

Exemple

Tous mes paragraphes en bleu et tous mes titres h1 en rouge

```
P{
  Color :blue ;
}
H1{
  Color : red ;
}
```

2.2. Appliquer un même style à plusieurs éléments

Lorsqu'un même style doit être appliqué à différents éléments, il suffit de combiner la déclaration en séparant les noms des éléments par une virgule, comme ceci :

```
Elément1, Elément2, .... {
  propriété: valeur;
  propriété: valeur;
  ....
}
```

Exemple :

Les titres h1, h2 et h3 en rouge et encadrés avec une bordure continue de 1 pixel en bleu.

```
h1{
  color: red;
  border : 1px solid blue ;
}
H2{
  color: red;
  border : 1px solid blue
}
H3{
  Color : red ;
  border : 1px solid blue
}
```

OU plus simplement :

```
h1,h2,h3{
  color: red;
  border : 1px solid blue ;
}
```

2.3. Utilisation des attributs class et id pour modifier que certaines occurrences d'un élément donné

Supposons maintenant qu'une page HTML contienne plusieurs paragraphes <p> et que certains d'entre eux doivent avoir une apparence différente. Pour arriver au but recherché, On pourrait placer le code CSS dans un attribut **style** sur la balise que l'on vise mais, comme cette technique n'est pas recommandée, il faudra utiliser les attributs spéciaux **id** et **class** permettant de donner un nom spécifique à une balise donnée c'est-à-dire un nom capable de différencier les balises d'un même type.

Remarque

Les attributs class et id fonctionnent sur toutes les balises et quasiment identique à une différence près :

- **L'attribut class** : il peut être utilisé autant de fois dans le code.
- **L'attribut id** : il ne peut être utilisé en réalité que sur des éléments qui sont uniques sur la page tel que le logo.

Mise en œuvre

Le principe est le même pour les deux attributs : Nommer la balise visée à l'aide de class ou de id et ensuite écrire le code CSS en fonction de l'attribut utilisé :

- Au cas où, vous utilisez l'attribut **class** pour nommer la balise, le code css prend la forme :

```
.nom{  
    Propriété : valeur ;  
    .....  
}
```

- Au cas où, vous utilisez l'attribut **id** pour nommer la balise, le code css prend la forme :

```
#nom{  
    Propriété : valeur ;  
    .....  
}
```

Exemple

Travail à faire :

Le 1^{er} et le 3^e paragraphe en bleu. Le 1^{er} titre h2 en vert

```
<!DOCTYPE html>
```

```
<html>
```

```
  <head>
```

```
    <meta charset="utf-8" />
```

```
    <link rel="stylesheet" href="style. css" />
```

```
    <title>nom site</title>
```

```
  </head>
```

```
  <body>
```

```
    <h1>cours de développement web</h1>
```

```
    <p class="vert">le développement web est la création de site web... </p>
```

```
    <p>Pour commencer nous allons apprendre le HTML et ensuite le CSS</p>
```

```
    <p class="vert">nous ferons du JavaScript et du PHP plus tard</p>
```

```
    <h2 id="titre">les langages HTML et CSS</h2>
```

```
    <p>HTML sert à structurer une page web tandis que CSS ) la présentation</p>
```

```
    <h2>les langages JS et PHP</h2>
```

```
    <p>JS crée du dynamisme coté navigateur et PHP interagit avec une BDD</p>
```

```
  </body>
```

```
</html>
```

Code: CSS

```
.vert{
    color: blue;
}
#titre{
    color: green;
}
```

2.4. Les sélecteurs avancés

- **Sélecteur universel : ***

Sélectionne tous les éléments sans exception. Sa syntaxe est la suivante :

```
*{
    Propriété1 : valeur ;
    ...
}
```

- **Sélectionner un élément contenu dans un autre**

```
Elément1 élément2{
    Propriété1 : valeur ;
    ...
}
```

NB : les élément1 et élément2 sont séparés par un espace

Exemple

```
<h3>Titre avec <em>texte important</em></h3>
```

Travail à faire : Tous les éléments **em** situés à l'intérieur d'un élément **h3 en rouge**.

```
h3 em {
    Color :red ;
}
```

- **Sélectionner un élément qui en suit un autre**

```
Elemnt1 + élément2 {
    Propriété1 : valeur ;
    ...
}
```

Exemple:

```
<p class="vert">nous ferons du JavaScript et du PHP plus tard</p>
```

```
<h2 id="titre">les langages HTML et CSS</h2>
```

```
<p>HTML sert à structurer une page web tandis que CSS ) la présentation</p>
```

Travail à faire : la première balise **<p>** située après un titre **<h2> en vert**

```
H2 + p{
    Color : green ;
}
```

- **Sélectionner un élément qui possède un attribut donné**

```
Element[ attribut]{
    propriété1 : valeur ;
    ....
}
```

Exemple:

Code : HTML

```
<a href="http: //site. com" title="Infobulle">  
<a href="info.html">Informations</a>  
<a href="partenaire.html">Nos Partenaires</a>
```

Travail à faire : tous les liens **a>** qui possèdent un attribut title rouge.

Code : CSS

```
a[ title] {  
    Color :red ;  
}
```

- **Sélectionner une balise ayant un attribut et une valeur exacte**

```
Elément[ attribut="valeur"] {  
    propriété : valeur ;  
    ...  
}
```

Exemple :

Code : HTML

```
<a href="http: //site. com" title="Cliquez ici">  
<a href="http: //site. com" title="Cliquez simplement">  
<a href="http: //site. com" title="Veuillez cliquer">
```

Travail à faire : toutes les balises **<a>** ayant un attribut title qui a pour valeur Cliquez ici en vert.

Code css :

```
a[ title="Cliquez ici"] {  
    color :green ;  
}
```

II. Formatage du texte en css

1. Police du texte

Pour choisir la police employée dans un 2L2MENT, vous utiliserez la propriété CSS **font-family** dont la syntaxe est la suivante:

```
Elément{  
    font-family: police1, ..., policeN;  
}
```

Remarque

Lorsque plusieurs polices sont spécifiées, la première police disponible sur l'ordinateur où s'exécute le code est utilisée. Vous devez choisir en dernier lieu un nom de police générique, qui existera forcément sur tous les ordinateurs : serif, sans-serif, cursive, fantasy ou monospace.

Voici quelques types de polices correspondant à chacun des types génériques :

- serif : Times New Roman, Bodoni, Garamond, MS Georgia.

- sans-serif : MS Trebuchet, MS Arial, MS Verdana, Univers, Futura, Gill Sans, Helvetica.
- cursive : Script, Segoe Script, Comic Sans MS, Viner Hand ITC, Vladimir Script.
- fantasy : Kino MT, Tektonpro, Critter, Studz.
- monospace : Courier, MS Courier New, Prestige.

Exemple : En CSS, dans une feuille de styles interne ou externe :

```
p {font-family: "Courier New", Courier, monospace;}
h1 {font-family: "Times New Roman", Garamond, serif;}
p {font-family: "Comic sans MS", Script, cursive;}
```

NB : Les noms de polices contenant des espaces doivent être placés entre guillemets. Il est fortement déconseillé de choisir une police fantasy pour afficher vos textes. En effet, chaque système d'exploitation possède ses propres polices fantasy. L'affichage est donc des plus aléatoires.

2. Taille des caractères

La taille des polices est choisie avec la propriété **font-size**. Les valeurs peuvent être spécifiées en :

- Une **taille absolue** : en inches (in), centimètres (cm), millimètres (mm), points (pt), picas (pc), pixels (px). Cette méthode est très précise mais il est conseillé de ne l'utiliser que si c'est absolument nécessaire, car on risque d'indiquer une taille trop petite pour certains lecteurs. (À titre d'information, 1 point = 1/72 pouce, 1 pica = 12 points, 1 pouce = 2,54 cm,)
- Une **taille relative** : en em, en pourcentages (%) ou en X-height (ex). cette technique a l'avantage d'être plus souple. Elle s'adapte plus facilement aux préférences de taille des visiteurs. (1 em correspond grossièrement à la hauteur de la lettre "Ç" et 1 ex à la hauteur de la lettre minuscule "x")

Il y a plusieurs moyens d'indiquer une valeur relative :

- **écrire la taille avec des mots en anglais** comme ceux-ci :
 - xx-small** : minuscule ;
 - x-small** : très petit ;
 - small** : petit ;
 - medium** : moyen ;
 - large** : grand ;
 - x-large** : très grand ;
 - xx-large** : très très grand
- **indiquer la taille en « em »**. avec 1em, le texte a une taille normale. Pour grossir le texte inscrire une valeur supérieure à 1 (ex 1. 3em) et pour le réduire inscrivez une valeur inférieure à 1 (ex 0. 8em).
- Indiquer la taille en % : 100% est un texte en taille normale, plus de 100% est plus gros et moins de 100% est moins gros.

Exemple: taille de police de notre page à 1.2em

```
body {
  font-size: 1.2em;
}
```

3. Attributs des caractères

- La propriété **font-weight** définit la graisse des caractères. Les valeurs possibles sont normal (valeur par défaut), bold, bolder, lighter, 100, 200, 300, 400, 500, 600, 700, 800, ou 900.

Exemple d'utilisation:

Les caractères en gras dans toute la page

```
body {  
    font-weight: bolder;  
}
```

- La propriété **font-style** détermine l'orientation de la police. Elle peut être initialisée avec trois valeurs : normal, italic et oblique, si la police admet cette valeur).

Exemple d'utilisation :

Le paragraphe nommé Par1 en italique

```
#par1 {  
    font-style: italic;  
}
```

- La propriété **text-decoration** détermine le soulignement ou le surlignement d'un élément. Les valeurs possibles sont les suivantes:
 - none : aucune décoration.
 - underline : caractères soulignés.
 - overline : caractères surlignés.
 - blink : caractères clignotants (sauf sous Internet Explorer).
 - line-through : caractères barrés.

Exemple d'utilisation:

Le titre principal h1 souligné

```
h1 {  
    text-decoration: underline;  
}
```

- La propriété **text-transform** permet de mettre un texte en minuscules ou en majuscules. Les valeurs possibles sont les suivantes :
 - none : aucune transformation.
 - capitalize : première lettre des mots en majuscule.
 - uppercase : tous les caractères en majuscules.
 - lowercase : tous les caractères en minuscules.

Exemple d'utilisation: le titre principal h1 en majuscule

```
h1 {  
    text-transform: uppercase;  
}
```

- La propriété **font-variant** permet de mettre un texte en petites majuscules lorsque la valeur small-caps lui est affectée

Exemple d'utilisation: texte de mes paragraphe en small caps

```
P {  
    font-variant: small-caps;  
}
```

- La propriété **text-indent** définit l'indentation de la première ligne d'un paragraphe. La valeur de l'indentation peut être spécifiée en utilisant une unité de taille quelconque (px, pt, mm, in, etc.) ou un pourcentage. Si la valeur est négative, le texte est décalé vers la gauche. Dans ce cas, pensez à définir une marge à gauche : sinon, le début du texte risque d'être masqué.

Exemple d'utilisation: indenté le paragraphe nommé ti de 30px

En HTML : `<p class="ti">Ce paragraphe est indenté sur la première ligne</p>`

En CSS :

```
#ti {
    text-indent: 30px;
}
```

III. Couleur des éléments et transparence

1. Couleur du texte

La couleur du texte est définie par la propriété CSS **color**.

Voici sa syntaxe :

```
Élément{
    color: couleur;
}
```

Où **couleur** peut être spécifiée :

- **Sous la forme d'un nom normalisé** : white, silver, gray, black, red, maroon, lime, green, yellow, olive, blue, navy, fuchsia, purple, aqua, teal.

Exemple d'utilisation : tous les paragraphes en bleu

```
p {
    color: blue;
}
```

- À l'aide d'un **code hexadécimal** sur 6 digits : #RRVVBB (où RR, VV et BB sont les composantes rouge, verte et bleue de la couleur, codées en hexadécimal entre 00 et FF). C'est une combinaison de lettres et de chiffres qui indiquent une couleur. On doit toujours commencer par écrire un dièse (#), suivi de six lettres ou chiffres allant de 0 à 9 et de A à F.

Ces lettres ou chiffres fonctionnent deux par deux. Les deux premiers indiquent une quantité de rouge, les deux suivants une quantité de vert et les deux derniers une quantité de bleu. En mélangeant ces quantités (qui sont les composantes Rouge- Vert-Bleu de la couleur) on peut obtenir la couleur qu'on veut. Ainsi, #000000 correspond à la couleur noire et #FFFFFF à la couleur blanche.

Exemple d'utilisation :

En HTML : `<span>je suis étudiant en IDA</span>`

En CSS : `span { background-color: #01B0F0; }`

- À l'aide de la fonction **RGB(R,G,B)** : La fonction RGB(R, G, B) permet de définir une couleur en fonction de ses composantes rouge, verte et bleue. Cette fonction s'applique à n'importe quelle propriété dont la valeur est une couleur : color, background-color, border-color, etc.

La fonction RGB() admet trois paramètres : RGB(red, green, blue) Où red, green et blue sont les composantes rouge, verte et bleue de la couleur. Ces trois informations peuvent prendre 256 valeurs, codées entre 0 et 255.

Exemples d'utilisation en HTML

Les paragraphes ont une couleur d'arrière-plan jaune;

```
p{
    background-color:rgb(255, 255, 0);
}
```

- À l'aide de la fonction **RGBA**(R,V,B,A) : La fonction RGBA() rend plus ou moins transparentes les couleurs manipulées dans le code CSS3. Cette fonction s'applique à n'importe quelle propriété dont la valeur est une couleur : color, background-color, border-color, etc.

La fonction RGBA() admet quatre paramètres : RGBA(red, green, blue, alpha) Où : **red**, **green** et **blue** sont les composantes rouge, verte et bleue de la couleur (Ces trois informations peuvent prendre 256 valeurs, codées entre 0 et 255) et **alpha** est le degré d'opacité de la couleur. Cette information est un nombre décimal codé entre 0 (transparent) et 1 (opaque).

Exemples d'utilisation en HTML

Les paragraphes ont une couleur d'arrière-plan jaune à moitié transparent;

```
p{
    background-color:rgba(255, 255, 0 ,0.5);
}
```

2. Couleur d'arrière-plan d'un élément

La couleur d'arrière-plan d'un élément est définie par la propriété CSS **background-color**.

Voici sa syntaxe :

```
Element{
    background-color: couleur;
}
```

Où **couleur** peut être spécifié :

- Sous la forme d'un nom normalisé
- À l'aide d'un code hexadécimal sur 6 digits
- À l'aide de la fonction RGB (R,V,B) , ou RGBA(R,V,B,A).

Exemple d'utilisation:

En HTML :

```
<div id="div1" >Texte dans la balise <div> </div>
<div id="div2" >Texte dans la balise <div> </div>
<div id="div3" style="opacity: 0.5;" >Texte dans la balise <div> </div> En
```

CSS :

```
#div1 { background-color:rgba(0, 0, 200, 1); }
#div2 { background-color:rgba(0, 0, 200, 0.5); }
#div3 { background-color:rgb(0, 0, 200); }
```

IV. Personnalisation des listes

Il est possible de personnaliser le type des puces ou des numéros dans une liste, en utilisant la propriété CSS **list-style-type**.

Elle peut prendre pour valeur : disc, circle, square, decimal, lower-roman, upper-roman, lower-alpha, upper-alpha, none.

Exemple de code CSS:

```
ol {
    list-style-type: lower-latin;
}
ul {
    list-style-type: square;
}
```

Remarque

Si vous vous sentez limité par les styles de puces accessibles par défaut en CSS3, sachez que la propriété **list-style-image** permet d'utiliser n'importe quelle image comme symbole de puce. Sa syntaxe est :

```
Elément {
    list-style-image: url(adresse);
}
```

Où **adresse** est l'adresse URI de la puce, au format GIF ou JPEG.

NB : Pour trouver des puces librement téléchargeables, rendez-vous sur la page http://createafreewebsite.net/free_bullets.html. Cliquez du bouton droit sur l'une des puces proposées et sélectionnez Enregistrer l'image sous dans le menu pour l'enregistrer sur votre disque dur. Il ne vous reste plus qu'à affecter l'URI de la puce à la propriété **list-style-image** pour obtenir l'effet recherché.

Exemple

```
<!DOCTYPE html>
<html>
  <head>
    <meta charset="UTF-8" />
    <title>Liste à puces</title>
    <style type="text/css" >
      ul {list-style-image: url(blueball.gif);}
    </style>
  </head>
  <body>
    <ul>
      <li>Premier</li>
      <li>Deuxième</li>
      <li>Troisième</li>
      <li>Quatrième</li>
    </ul>
  </body>
</html>
```

V. Mise en forme des liens hypertextes

1. changer la couleur de lien non visité

Par défaut, les liens hypertexte apparaissent en caractères bleu soulignés lorsqu'ils n'ont pas été visités. Pour choisir d'autres couleurs, vous utiliserez la propriété CSS **color**:

```
a {  
    color: valeur;  
};
```

2. Autres propriétés CSS applicables aux liens

- Le type de soulignement peut également être choisi avec la propriété CSS **text-decoration** :
 - a {text-decoration: valeur;} pour les liens non visités ;
 - a:visited {text-decoration: valeur} pour les liens visités.
- Les propriétés relatives au texte sont également utilisables dans un style relatif aux liens hypertexte. Vous utiliserez en particulier les propriétés suivantes :
 - font-family : police(s) à utiliser.
 - font-size : taille des caractères.
 - font-style : orientation des caractères (normal, italic ou oblique).
 - font-weight : graisse de la police.
 - font-variant : caractères par défaut (normal) ou petites majuscules (small-caps).
 - text-transform : casse des caractères (none, capitalize, uppercase ou lowercase).
 - background-color : couleur d'arrière-plan du lien.

Exemple

Cet exemple utilise deux classes pour illustrer les différentes couleurs et décorations que peuvent revêtir les liens non visités et les liens visités.

Code html

```
<!DOCTYPE html>  
<html>  
  <head>  
    <meta charset="UTF-8" />  
    <title>Mise en forme des liens hypertexte</title>  
    <link rel="stylesheet" href="style.css">  
  </head>  
  <body>  
    <a href="http://www.page1.fr" class="type1" >Lien vers la page 1</a>  
    <a href="http://www.page2.fr" class="type2" >Lien vers la page 2</a>  
  </body>  
</html>
```

Code du fichier CSS : style.css

```
.type1 {
    color: red;
    text-decoration: none;
}
.type1:visited {
    color: blue;
    text-decoration: underline;
}
.type2 {
    color: blue;
    text-decoration: overline;
}
.type2:visited {
    color: red;
    text-decoration: none;
}
```

VI. Pseudo formats et apparences dynamiques

Le CSS permet aussi de modifier l'apparence des éléments de façon dynamique grave aux pseudo-formats. Un pseudo-format est une information que l'on rajoute après le nom de la balise (ou de la classe). Le nom de la balise ou de la classe est suivi de deux points (:) et du nom du pseudo format.

Les pseudos formats permettent ainsi de changer l'apparence de nos différents éléments **au survol**, **lors du clic**, **lorsqu'un élément a le focus** (sélectionné) ou **lorsqu'un lien a été consulté**.

1. Changer l'apparence d'un élément au survol

Pour changer l'apparence d'un élément au survol, on utilise le pseudo format **:hover**.

```
Elément:hover {
    Propriété1 : valeur ;
    .....
}
```

Exemple : changer la couleur d'un lien au survol

```
a:hover{
    Color : gray ;
}
```

2. Changer l'apparence d'un élément au moment du clic

Pour changer l'apparence d'un élément au moment du clic, on utilise le pseudo format **:active**. En pratique, il n'est utilisé que sur les liens. Le lien gardera alors cette apparence très peu de temps : en fait, le changement intervient lorsque le bouton de la souris est enfoncé.

```
Elément :active{
    Propriété1 : valeur ;
    .....
}
```

Exemple : changer la couleur de fond du lien lorsque l'on clique dessus :

```
a: active{
    background-color: rgb(120,50,240);
}
```

3. Changer l'apparence d'un élément lorsqu'il est sélectionné

Pour changer l'apparence d'un élément lorsqu'il est sélectionné, on utilise le pseudo format **:focus**. On l'utilise le plus souvent sur les liens mais il peut être appliqué à d'autres balises.

```
Elément :focus{
    Propriété1 : valeur ;
    .....
}
```

Exemple : changer la couleur de fond du lien lorsque l'utilisateur le sélectionne.

```
a:focus{
    background-color: green;
}
```

NB : Sous Google Chrome et Safari, l'effet ne se voit que si l'on appuie sur la touche Tab.

4. Changer l'apparence d'un lien lorsqu'il a été consulté (visité)

Par défaut, les liens hypertexte visités sont affichés en caractères bordeaux souligné. Pour choisir d'autres couleurs pour un lien visité, vous utiliserez le pseudo-format :

visited

```
a :visited {
    Propriété : valeur ;
    .....
}
```

Exemple:

```
a:visited{
    color: navy;
}
```

VII. Gestion des images en css

Dans cette section, nous allons modifier l'image de fond de la page. Cependant, tout comme pour la couleur de fond, l'image de fond ne s'applique pas forcément à la page entière. On peut aussi mettre une image de fond derrière les titres, paragraphes, etc.

1. Image en arrière-plan d'une page

Pour insérer une image en arrière-plan d'une page Web, on utilise la propriété **background-image** s'appliquer à l'élément **html** ou sur l'élément **body**. Comme valeur, on doit renseigner **url (nom_image.format)**. Sa syntaxe est la suivante :

```
Elément{ Background-image :url(adresse_image) ;}
```

Où **adresse_image** désigne l'emplacement et le nom de l'image.

1.1. Options disponibles pour l'image de fond

On peut compléter la propriété **background-image** par plusieurs autres propriétés qui permettent de changer le comportement de l'image de fond :

- **background-attachment : fixer le fond**

La propriété **background-attachment** permet de « fixer » le fond. Ainsi le texte « glisse » par-dessus le fond. Deux valeurs sont disponibles: **fixed** (l'image de fond reste fixe), **scroll** (l'image de fond défile avec le texte (par défaut)).

Exemple:

```
body{
    background-image: url( "neige. png") ;
    background-attachment: fixed; /* Le fond restera fixe */
}
```

- **background-repeat : répétition du fond**

Par défaut, l'image de fond est répétée en mosaïque. Vous pouvez changer cela avec la propriété **background-repeat**. Les valeurs possibles de cette propriété sont:

- no-repeat : le fond ne sera pas répété. L'image sera donc unique sur la page.
- repeat-x : le fond sera répété uniquement sur la première ligne, horizontalement.
- repeat-y : le fond sera répété uniquement sur la première colonne, verticalement.
- repeat : le fond sera répété en mosaïque (par défaut).

Exemple d'utilisation:

```
body{
    background-image: url(" soleil. png");
    background-repeat: no-repeat;
}
```

- **background-position : position du fond**

On peut indiquer la position de l'image de fond avec **background-position**. Cette propriété n'est intéressante que si elle est combinée avec **background-repeat: no-repeat**. Vous devez donner à **background-position** deux valeurs en pixels pour indiquer la position du fond par rapport au coin supérieur gauche de la page (ou de tout autre conteneur). La syntaxe est la suivante :

```
body{
    background-image: url(adresse_image);
    background-position: Xpx Ypx ;
}
```

Votre fond sera placé à X pixels de la gauche et à Y pixels du haut. Il est aussi possible d'utiliser ces valeurs en anglais:

- top : en haut ;
- bottom : en bas ;
- left : à gauche ;
- center : centré ;
- right : à droite.

Remarque

Il est possible de combiner ces mots. Par exemple, pour aligner une image en haut à droite, vous taperez: **background-position: top right;**

Exemple:

Si je veux afficher un soleil en image de fond, en un unique exemplaire (no repeat), toujours visible.

```
body{
    background-image : url( "soleil. png") ;
    background-attachment: fixed; /* Le fond restera fixe */
    background-repeat: no-repeat; /* Le fond ne sera pas répété */
    background-position: top right; /* Le fond sera placé en haut à droite */
}
```

1.2. Combiner les propriétés

Si vous utilisez beaucoup de propriétés en rapport avec le fond, vous pouvez utiliser une sorte de « super-propriété » appelée **background** dont la valeur peut combiner plusieurs des propriétés vues précédemment : background-image, background-repeat, background-attachment et background-position.

On peut donc tout simplement écrire :

Exemple :

```
body{
    background: url( "soleil. png") fixed no-repeat top right;
}
```

NB :

L'ordre des valeurs n'a pas d'importance et on n'est pas obligés de mettre toutes les valeurs.

2. Plusieurs images de fond

Depuis CSS3, il est possible de donner plusieurs images de fond à un élément. Pour cela, il suffit de séparer les déclarations par une virgule, comme ceci:

```
body{
    background: url ( "ci. png") fixed no-repeat top right, url ( "elephant. jpg") fixed;
}
```

Remarque :

La première image de cette liste sera placée par-dessus les autres.

L'ordre de déclaration des images a son importance.

3. Empiler des images en CSS

La propriété **z-index** permet de spécifier la position relative de plusieurs éléments empilés. Plus z-index est élevé, plus l'élément se trouve en avant-plan. Inversement, plus z-index est faible, plus l'élément se trouve en arrière-plan.

Dans cet exemple, nous superposons quatre images en jouant sur leur z-index.

Lorsqu'une image est pointée, son z-index est initialisé à 100 pour la faire passer en avant-plan.

Code html

```
<!DOCTYPE html>
<html>
  <head>
    <meta charset="UTF-8" />
    <title>Empilement d'images avec z-index</title>
    <link rel="stylesheet" href="style.css">
  </head>
  <body>
    
    
  </body>
</html>
```

Code CSS « style.css »

```
img {
  position: absolute;
}
#ci {
  top: 20px;
  left: 20px; z-index: 1;
}
#elephant {
  top: 90px;
  left: 70px; z-index: 2;
}
#ci:hover {
  z-index: 100;
}
#elephant:hover {
  z-index: 100;
}
```

VIII. Ordures et ombres en CSS

Le CSS3 apporte de nombreuses améliorations dans la mise en forme des éléments HTML. En particulier en ce qui concerne l'ombrage et les coins arrondis. Ce chapitre va vous montrer comment ombrer des éléments textuels et non textuels et comment ajouter des ombres sur des éléments textuels et non textuels.

1. Ombrage des objets non textuels

L'ombrage des éléments HTML5 non textuels se fait via la propriété CSS3 **box-shadow**. La syntaxe est la suivante :

```
Elément{  
    box-shadow: offet-horiz offet-vert rayon-ombrage couleur;  
}
```

Où:

- *offet-horiz* est le décalage horizontal de l'ombrage en pixels.
- *offet-vert* est le décalage vertical de l'ombrage en pixels.
- *rayon-ombrage* est le rayon de l'ombrage en pixels.
- *couleur* est la couleur de l'ombrage.

Cette propriété présente deux avantages : elle est compatible avec la quasi-totalité des navigateurs actuels (y compris Internet Explorer) et elle s'applique à tous les éléments HTML5.

Exemple:

CODE HTML

```
<!DOCTYPE html>  
<html>  
    <head>  
        <meta charset="UTF-8" />  
        <title>Ombrage CSS3</title>  
        <link rel="stylesheet" href="style1.css">  
    </head>  
    <body>  
        <div>Texte à l'intérieur de la balise <div>  
    </div>  
    </body>  
</html>
```

CODE CSS « style1.css »

```
div {  
    width: 300px;  
    height: 300px;  
    background: yellow;  
    font-size: 400%;  
    box-shadow: 10px 10px 5px grey;  
}
```

Les instructions CSS définissent les dimensions, la couleur d'arrière-plan, la taille des caractères et les propriétés de l'ombrage.

Remarque:

- Pour que l'ombrage soit plus diffus, il suffit d'augmenter son rayon. `box-shadow: 10px 10px 25px grey;`
- En inversant le signe des deux premiers paramètres de la propriété `box-shadow`, on peut changer la position de la bordure : `box-shadow: -10px -10px 25px grey;`
- La propriété `box-shadow` peut également être appliquée à une image. Il suffit pour cela de nommer votre image à l'aide de l'attribut **class** et de lui appliquer un style comme par exemple :

EN HTML

```

```

EN CSS

```
.image {  
    box-shadow: 10px 10px 25px grey;  
}
```

2. Ombrage des objets textuels

L'ombrage CSS3 d'éléments textuels se fait via la propriété **text-shadow** dont la syntaxe est la suivante :

```
Elément{  
    text-shadow: offset-horiz offset-vert épaisseur couleur;  
}
```

Où :

- `offset-horiz` est le décalage horizontal de l'ombrage en pixels.
- `offset-vert` est le décalage vertical de l'ombrage en pixels.
- `épaisseur` est l'épaisseur de l'ombrage en pixels.
- `couleur` est la couleur de l'ombrage.

Exemple d'utilisation de cette propriété

CODE HTML

```
<!DOCTYPE html>  
<html>  
    <head>  
        <meta charset="UTF-8" />  
        <title>Ombrage de texte en CSS3</title>  
        <link rel="stylesheet" href="style2.css">  
    </head>  
    <body>  
        <p class="otexte" >Ce texte est ombré</p>  
    </body>  
</html>
```

CODE CSS « style2.css »

```
p{  
    font-family: magneto;  
    font-size: 60px;  
    text-shadow: 4px 4px 4px grey;  
}
```

3. Arrondir les coins d'un élément de type block

Pour définir des arrondis sur un élément de type block, vous utiliserez la propriété CSS3 **border-radius** (Avec certains navigateurs, il n'est pas nécessaire d'utiliser des préfixes) dont la syntaxe est la suivante:

```
Elément{  
    border-radius: r1 r2 r3 r4;  
}
```

Où r1, r2, r3 et r4 représentent respectivement le rayon des coins supérieur gauche, supérieur droit, inférieur droit et inférieur gauche de l'élément.

Si un seul rayon est spécifié, il s'applique aux quatre coins de l'élément.

Exemple :

Définir une bordure noire épaisse de 1 pixel autour d'une balise <div> et arrondit ses quatre angles avec un rayon de courbure de 20 pixels:

```
<!DOCTYPE html>  
<html>  
    <head>  
        <meta charset="UTF-8" />  
        <title>Une boîte à coins arrondis en CSS3</title>  
        <style>  
            .arrondi {  
                border-radius: 20px;  
                border: 1px solid black;  
            }  
        </style>  
    </head>  
    <body>  
        <div class="arrondi" >Boîte à bords arrondis en CSS3</div>  
    </body>  
</html>
```

Comme vous pouvez le constater, l'arrondi s'étend sur toute la largeur de la page, car la balise <div> n'a pas été dimensionnée. Il est possible de limiter ses dimensions avec les propriétés **width** et **height**. D'autre part, le texte à l'intérieur de la balise <div> est un peu trop collé sur le bord gauche. Pour résoudre ce problème, il suffit de définir un **padding**. Pour finaliser la mise en forme, vous pouvez également modifier l'alignement du texte avec la propriété **text-align** et colorer l'arrière-plan avec la propriété **background**.

Voici le code CSS3 remanié:

```
.arrondi {  
    border-radius: 15px;  
    border: 1px solid black;  
    padding: 10px;  
    width: 250px;  
    height: 200px;  
    background:#ADCF4F;  
    text-align: center;  
}
```

L'arrière-plan peut également être une image, répétée si nécessaire pour couvrir toute la surface de l'élément. Ici, par exemple, nous avons pris l'image fond.gif et nous l'avons répétée pour couvrir toute la surface de la balise <div>. La taille des caractères a également été modifiée, ainsi que la hauteur de la balise <div> et l'épaisseur de la bordure.

```
.arrondi{
    border-radius: 15px;
    border: 5px solid black;
    padding: 10px;
    width: 250px;
    height: 300px;
    font-size: 60px;
    background: url("fond.gif") repeat;
    text-align: center;
}
```

Dans ce dernier exemple, nous appliquons des arrondis différents sur un élément div pour lui donner une forme un peu plus évoluée.

```
<!DOCTYPE html>
<html>
  <head>
    <meta charset="UTF-8" />
    <title>Une boîte à coins arrondis en CSS3</title>
    <style>
      .arrondi {
        border-radius: 50px 50px 200px 50px;
        border: 1px solid black;
        width: 350px;
        height: 150px;
        padding-top:
        50px; padding-
        left: 20px;
      }
    </style>
  </head>
  <body>
    <div class="arrondi" >Boîte à bords arrondis en CSS3</div>
  </body>
</html>
```

IX. Le modèle des boîtes

Une page web peut être vue comme une succession et un empilement de boîtes, qu'on appelle « blocs ». Dans ce chapitre, nous allons à manipuler ces blocs en leur donnant des dimensions, en les agençant en jouant sur leurs marges et à gérer leur contenu pour éviter que le texte ne dépasse de ces blocs.

1. Les balises de type block et inline

En HTML, la plupart des balises peuvent se ranger dans l'une ou l'autre de deux catégories:

- Les balises **inline** : une balise de type inline se trouve obligatoirement à l'intérieur d'une balise block. Une balise inline ne crée pas de retour à la ligne, le texte qui se trouve à l'intérieur s'écrit donc à la suite du texte précédent, sur la même ligne (ex : , , <mark>, <a>, , ...)
- Les balises **block** : une balise de type block sur une page web crée automatiquement un retour à la ligne avant et après. La page web sera en fait constituée d'une série de blocs les uns à la suite des autres. Il est possible de mettre un bloc à l'intérieur d'un autre, ce qui va augmenter considérablement nos possibilités pour créer le design de notre site (ex :<footer>, <h1>, <h2>, <article>,...).

***NB** : Il existe plusieurs autres catégories très spécifiques, par exemple pour les cellules de tableau (type `tablecell`) ou les puces (type `list-item`).*

2. Les balises universelles

Il existe 2 balises qui n'ont aucun sens sémantique (comme contrairement à p qui veut dire paragraphe).

Le principal intérêt de ces balises est que l'on peut leur appliquer une class (ou un id) pour le CSS quand aucune autre balise ne convient.

- **** (inline) ;
- **<div></div>** (block)

***NB** : N'utilisez ces balises universelles que s'il n'y a pas de balises plus adaptées.*

3. Les dimensions

Contrairement à un inline, un bloc a des dimensions précises, possède une largeur et une hauteur. Nous disposons en CSS de deux propriétés permettant de modifier ces 2 entités :

- **width** : Largeur du bloc en pixels (px) ou en pourcentage (%).
- **height** : Hauteur du bloc en pixels (px), ou en pourcentage (%).

Pour être exact, **width** et **height** représentent la largeur et la hauteur du contenu des blocs. Si le bloc a des marges, celles-ci s'ajouteront à la largeur et la hauteur.

Par défaut, un bloc prend 100% de la largeur disponible.

Exemple de code modifiant la largeur des paragraphes à 50% ou de 250 pixels.

Code : CSS

```
p{
    width: 50%;
}
Ou
p{
    width: 250px;
}
```

4. Marges

Toutes les balises affichant du contenu sur l'écran possèdent 2 types de marges :

- Des marges internes (L'espace entre le texte et la bordure).
- Des marges externes (L'espace entre la bordure et le bloc suivant).

Vous pouvez modifier ces marges en agissant sur les propriétés CSS **padding** (marges internes) et **margin** (marges externes). À titre d'exemple, le code suivant définit une balise <div> et y insère du texte.

```
<!DOCTYPE html>
<html>
  <head>
    <meta charset="UTF-8" />
    <title>Margin et padding</title>
  </head>
  <body>
    <div>
      Toutes les balises visuelles possèdent 2 types de marges : <br>
      -Des marges internes (L'espace entre le texte et la bordure). <br>
      -Des marges externes (L'espace entre la bordure et le bloc suivant). <br>
      Vous pouvez modifier ces marges en agissant sur les propriétés CSS
      padding (marges internes) et margin (marges externes).
    </div>
  </body>
</html>
```

CODE CSS

```
div {
  margin: 100px;
  padding: 50px;
  background-color: red;
  width: 300px;
  height: 250px;
  text-align: justify;
}
```

Remarque

Pour définir des marges pour chaque côté d'un bloc on peut utiliser les propriétés CSS suivantes :

- **margin-top** : marge extérieure en haut ;
- **margin-bottom** : marge extérieure en bas ;
- **margin-left** : marge extérieure à gauche ;
- **margin-right** : marge extérieure à droite.
- **padding-top** : marge intérieure en haut ;
- **padding-bottom** : marge intérieure en bas ;
- **padding-left** : marge intérieure à gauche ;
- **padding-right** : marge intérieure à droite.

Remarque

Il y a d'autres façons de spécifier les marges avec les propriétés **margin** et **padding**.
Par exemple:

- **margin: 2px 0 3px 1px;** signifie « 2 px de marge en haut, 0 px à droite (le px est facultatif dans ce cas), 3 px en bas, 1 px à gauche »
- **margin: 2px 1px;** signifie « 2 px de marge en haut et en bas, 1 px de marge à gauche et à droite ».

5. Centrer des blocs

Il est tout à fait possible de centrer des blocs. C'est même très pratique pour réaliser un design centré quand on ne connaît pas la résolution du visiteur.

Pour centrer, il faut respecter les règles suivantes :

- donnez une largeur au bloc (avec la propriété **width**) ;
- indiquez que vous voulez des marges extérieures automatiques, comme ceci :
margin: auto;

Code CSS

```
p{
    width: 350px; /* On a indiqué une largeur (obligatoire) */
    margin: auto; /* le bloc est centré avec auto */
    border: 1px solid black ;
    text-align: justify;
    padding: 12px;
    margin-bottom: 20px;
}
```

NB : Il n'est pas possible de centrer verticalement un bloc avec cette technique.
Seul le centrage horizontal est permis.

X. Tableaux et CSS

1. bordures des colonnes

La propriété CSS **border** accessible dans les éléments **table**, **th** et **td**, la propriété **border** permet (respectivement) de définir la bordure autour du tableau, des cellules de titre et des cellules de données. Voici sa syntaxe :

```
Elément{
    border: épaisseur style couleur;
}
```

Où :

- **épaisseur** est l'épaisseur de la bordure : thin, medium, thick ou une valeur numérique exprimée en pixels (px), points (pt), pourcentage (%), ou une autre unité de mesure CSS.
- **style** est le style de la bordure : dashed, dotted, double, solid, insert, groove, ridge ou outset.
- **couleur** est la couleur de la bordure : une couleur prédéfinie, ou une valeur hexadécimale ou RGB.

Exemple

```
table{
    border: 4px dashed black;
}
th, td{
    border: 1px solid red;
}
```

2. fusionner et séparer des bordures

La propriété CSS **border-collapse** détermine si les cellules d'un tableau sont séparées (valeur **separate**) ou fusionnées (valeur **collapse**). Elle s'applique à l'élément **table**.

Exemple :

```
table {
    border: 4px dashed black;
    border-collapse: collapse;
}
```

3. espacer les bordures

La propriété **border-spacing** détermine l'espace qui sépare les bordures des cellules adjacentes d'un tableau.

Exemple de code CSS

Nous ici définissons un tableau encadré par une bordure continue noire épaisse de 1 pixel. Les cellules (<td>) sont séparées (border-collapse: separate;) et espacées de 10 pixels (border-spacing: 10px;). Pour bien différencier les cellules du tableau, une couleur d'arrière-plan a été affectée aux balises <td> et une autre à la balise <table> : La propriété border-spacing détermine l'espace qui sépare les bordures des cellules adjacentes d'un tableau.

```
table {
    border: 1px solid black;
    text-align: center;
    border-collapse: separate;
    border-spacing: 10px;
    background-color: #CFF4EF;
}
td {
    width: 120px;
    border: 1px solid red;
    background-color: #F4CFD7;
}
```

4. Arrière-plan des cellules

L'arrière-plan des cellules peut être une couleur uniforme ou une image. Dans le premier cas, vous utiliserez la propriété **background-color** et, dans le second, la propriété **background: url**.

5. Préciser l'emplacement de la légende

Par défaut, la légende est affichée au-dessus du tableau et centrée horizontalement sur le tableau. Cependant, vous pouvez utiliser la propriété CSS **caption-side** applicable à l'élément **caption** pour préciser l'emplacement de la légende.

Cette propriété peut prendre les valeurs suivantes :

- **top** : légende au-dessus du tableau (valeur par défaut).
- **bottom** : légende en dessous du tableau.
- **left** : légende à gauche du tableau.
- **right** : légende à droite du tableau.

Bien entendu, vous pouvez fixer un ou plusieurs autres arguments dans le style de l'élément **caption**, en particulier l'alignement du texte, la taille et les attributs des caractères.

Exemple :

```
caption {  
    caption-side: bottom;  
    font-style: italic;  
    text-align: right;  
}
```

6. Alignement dans les cellules

Vous utiliserez les propriétés CSS **text-align** et **vertical-align** pour définir (respectivement) l'alignement horizontal et vertical des cellules **th** et **td** dans un tableau.

- La propriété **text-align** peut prendre les valeurs **left** (gauche), **right** (droite), **center** (centré) ou **justify** (justifié).
- La propriété **vertical-align** peut prendre les valeurs **top** (alignement sur la partie supérieure de la cellule), **bottom** (alignement sur la partie inférieure de la cellule) ou **middle** (centré sur la hauteur de la cellule).

XI. POSITIONNEMENT EN CSS